

Asynchronous, Generation-Free ABC: Streaming AMIS Reweighting for Heterogeneous Simulator Workloads on HPC

First Author^{1*} and Second Author¹

^{1*}Department, Organization, City, Country.

*Corresponding author(s). E-mail(s): first.author@example.com;
Contributing authors: second.author@example.com;

Abstract

Sequential approximate Bayesian computation advances in generations, and a generation cannot end before its slowest simulation. On a parallel machine that barrier idles every worker that finished early, at a cost that for identically distributed simulation times is the straggler factor of the workload: the expected maximum of W times over their mean. We introduce a generation-free ABC sampler that updates its proposal after every arriving evaluation, weights every evaluated particle by a streaming form of adaptive multiple importance sampling, and reports an estimator that is a pure function of the evaluated history. We prove the reported estimator consistent up to a fidelity ratio we decompose and partly measure, verify its calibration, and measure four things. (i) The barrier's cost, predicted from the asynchronous run's timing, matches a barrierized twin of our sampler within **1%** under a persistent straggler, **20%** under lognormal heterogeneity, and **10%** on a Cellular Potts tissue simulator, from **1.2×** to **400×**. (ii) Removing it converts into a tighter tolerance: on the tissue simulator the asynchronous sampler completes **2.1–2.4×** the simulations of a matched synchronous baseline (**2.0×** of it the barrier's), is **1.6×** more efficient per simulation, and ends **4.1×** tighter at equal wall clock. (iii) The throughput advantage turns on at a few milliseconds per simulation. (iv) The reported posterior recovers a two-parameter tissue posterior once the starting bandwidth is set from the prior-predictive discrepancy scale; its effective sample size, a few multiples of the archive size, is the remaining limit.

Keywords: approximate Bayesian computation, sequential Monte Carlo, adaptive multiple importance sampling, asynchronous algorithms, high-performance computing, likelihood-free inference

1 Introduction

Approximate Bayesian computation (ABC) is the standard route to Bayesian inference when a simulator can be run but its likelihood cannot be evaluated [1]. Its sequential forms — ABC-SMC and ABC-PMC [2–5] — move a population of particles through a sequence of shrinking tolerances, and every implementation we know of does so in generations: the proposal and the tolerance are updated once the whole population has been simulated. That is a synchronization barrier. On a parallel machine with W workers, a generation ends when its slowest simulation does, and every worker that finished earlier waits. The cost is easy to state: if simulation times T_i are drawn from a distribution with mean μ , a generation of W takes about $\mathbb{E}[\max_{i \leq W} T_i]$, so the barrier costs a factor $\mathbb{E}[\max_{i \leq W} T_i]/\mu$ — the *straggler factor* of the workload. It is 1 for a homogeneous simulator and grows with W and with the spread of the runtime distribution. In general the cost is the ratio of the barrier-free throughput to $W/\mathbb{E}[\max_{i \leq W} T_i]$, which is what we predict and measure; it reduces to the straggler factor when every worker draws from the same distribution and the barrier-free arm runs at W/μ . Simulators that are worth running on such machines are rarely homogeneous: the cost of a cell-based tissue simulation, for instance, tracks the number of cells it produces, which varies by orders of magnitude across a prior, and we measure its runtime coefficient of variation rising from 0.05 to above 0.2 as the simulated volume grows.

This paper removes the generation. We propose an ABC sampler in which the proposal mixture is rebuilt after every arriving evaluation, from a sliding archive of the k best particles evaluated so far; every evaluated particle is weighted against the mixture of proposals actually used — the streaming limit of adaptive multiple importance sampling (AMIS) [6] — and the estimator the method reports is a pure function of the evaluated history. Nothing has to be synchronized between workers, so the sampler runs on the barrier-free island model of the Propulate optimization engine [7], and a crashed run is recovered by replaying its log rather than by checkpointing sampler state. Look-ahead scheduling [8] attacks the same idle time by starting the next generation early on free workers; it keeps the generation, and recovers the 10–50% that the boundary wastes. We remove the boundary, and recover the factor.

Our contributions are:

1. **A generation-free, single-arrival-driven ABC sampler** combining smooth-kernel ABC with streaming AMIS reweighting, whose reported estimator is history-reconstructed and therefore replayable, with its implementation in Propulate (§3, §5).
2. **A predictive account of what the barrier costs.** We build a barrierized twin of our own sampler — identical in everything but a collective barrier before each proposal — and show that its slowdown is predicted from the asynchronous run’s timing, or the injected runtime law, alone: within 1% under a persistent straggler, within 20% under lognormal heterogeneity, and within 10% on a real tissue simulator at two sizes, over $1.2\times$ to $400\times$ (§7.1). The same prediction lets a production run estimate its own gain from timing data, without a wasteful reference run.
3. **The measurement that the gain converts, and where it turns on.** Against a pyABC baseline matched on kernel and bandwidth schedule, the tolerance reached

per simulation is at least the baseline’s on every benchmark but the one-dimensional analytic one, so throughput multiplies through: $2.1\text{--}2.4\times$ the simulations (the barrier’s share $2.0\times$), $1.6\times$ per simulation, $4.1\times$ tighter at equal wall clock on the tissue simulator (§7.2). The throughput ratio crosses one at a few milliseconds per simulation; below that the per-arrival coordination costs more than the barrier (§7.3).

4. Consistency and calibration of the reported estimator, and its limits.

We prove consistency, up to a fidelity ratio we decompose and partly measure, for the estimator the experiments report (§4; the proofs and a central limit theorem for damped-adaptation variants are in the supplement), verify calibration by simulation-based calibration in one and four dimensions and under multimodality, and recover a two-parameter Cellular Potts posterior on synthetic data with a known truth. Two settings govern what the estimator delivers: the starting bandwidth, which must be set from the prior-predictive discrepancy scale on an expensive simulator, and the archive size, which bounds the effective sample size at a few multiples of itself (§7.4).

Two limits are stated up front. On simulators cheaper than a few milliseconds the method loses to a synchronous one, and two of our four benchmarks lie on that side of the line by design. And the tissue benchmark is a calibration instrument, not a production run: its simulations cost seconds where production runs cost hours, so its role is to establish the method against a known truth and to show that its gain is predictable from timing, which is what a production run with an unknown posterior can use.

The paper is organized as follows. §2 gives the ABC and importance-sampling background, §3 the method, §4 its theory, §5 the implementation and the two instruments we compare against, §6 the benchmarks and metrics, §7 the results by claim, and §8–§10 discussion, limitations and conclusion.

2 Background and Related Work

2.1 Approximate Bayesian computation

ABC replaces exact likelihood evaluation with simulation and discrepancy-based acceptance. For observed data y and parameter θ , the ABC target at bandwidth ϵ is

$$\pi_\epsilon(\theta \mid y) \propto \pi(\theta) L_\epsilon(\theta), \quad L_\epsilon(\theta) = \mathbb{E}_{\rho \sim p(\rho|\theta)}[K_\epsilon(\rho)], \quad (1)$$

where ρ is the discrepancy between simulated and observed summaries, π is the prior, and K_ϵ is the ABC kernel. Classical ABC uses the hard indicator $K_\epsilon(\rho) = \mathbf{1}[\rho < \epsilon]$; *smooth-kernel* ABC [9, 10] replaces this with a continuous kernel (e.g. Gaussian $\exp(-\rho^2/2\epsilon^2)$ or Epanechnikov), which removes the discontinuity in the likelihood approximation and admits standard SMC-sampler theory.

2.2 Sequential ABC algorithms

The generation-staged family (ABC-SMC [2, 3], ABC-PMC [4], and adaptive variants [5, 11, 12]) advances through intermediate targets using mixture proposals

$$q_r(\theta) = \sum_{j=1}^{N_r} W_j^{(r)} K(\theta \mid \theta_j^{(r)}) \quad (2)$$

built from accepted particles of the previous stage. The defining constraint is the *generation barrier*: the importance weights at stage r are computed against a single proposal q_{r-1} , which requires the full stage- r population before q_r can be formed.

2.3 Adaptive multiple importance sampling

AMIS [6] generalizes staged importance sampling: at each stage the proposal is adapted from past particles, and crucially *all* past particles are re-weighted against the cumulative proposal mixture $\frac{1}{n} \sum_{j=1}^n q_{\tau_j}$ (the balance heuristic of [13]) rather than against the proposal under which each was originally drawn. AMIS is consistent under regularity conditions [14]; the convergence argument in [6] is heuristic. Our method takes the *streaming limit* of AMIS: stage size one, single-arrival-driven adaptation.

2.4 Asynchronous SMC and parallel ABC

Our proposal-adaptation machinery descends from the adaptive-importance-sampling line — population Monte Carlo [15], AMIS [6] and its recycling-consistency theory [14], surveyed in [16] — which we take to its streaming, single-arrival limit. Parallel and off-barrier SMC has been studied for state-space models — the island particle model [17], asynchronous anytime particle methods [18], and parallel resampling [19] — but none targets ABC, and none combines barrier-free execution with AMIS-style cumulative-mixture reweighting. Distributed ABC frameworks such as pyABC [20, 21], ABCpy [22], and ELFI [23] parallelize *simulation* effectively within a generation.

The closest work to ours is look-ahead scheduling [8], which attacks exactly the idle time we do. Rather than let workers stall while generation $t-1$ drains, it starts generation- t simulations preemptively on the free workers, from a proposal built either from the preliminary population of the first N acceptances (LA PREL) or from the last finalised population $\mathcal{P}_{t-2}$ (LA PAST), and importance-weights each preliminary draw against whichever proposal produced it. It also identifies, and corrects for, a runtime-dependent selection that we do not: sampling from the preliminary population favours parameters that simulate quickly, which the authors remove either by proposing from $\mathcal{P}_{t-2}$ or by closing a generation on the N accepted particles that were *started* first rather than the ones that finished first. That start-order rule is the object our proofs need and our implementation does not have (§9, Appendix B.5).

The difference is what is kept. Look-ahead fills the drain window but retains the generation: a population is still declared, a proposal is still fitted once per population, and the weight is still against a single per-generation proposal g_t . We have no populations, adapt after every arrival, and weight against the cumulative mixture of

all proposals used so far. The speed-ups differ in kind accordingly — theirs are the 10–50% recoverable from idle time at the boundary, ours are the factor the boundary itself costs (§7.1) — and the two are complementary rather than competing. What we take on in exchange is that standard generational SMC theory no longer applies, which is what §4 is for.

3 Method

3.1 Design constraints and history-based state

We design the algorithm so that the *reported estimator* is a pure function of the evaluated history: the archive, the reconstructed bandwidth and the posterior weights are all recomputed from the recorded log, so a crashed run is recovered by replaying that log rather than by checkpointing sampler state, and the estimator is identical whether it is formed online or afterwards.

That is a property of the estimator, not of the running sampler: the propagator does carry state between calls — the online AMIS snapshot buffer enters the denominator of every candidate it proposes, and two scheduler throttles gate when the bandwidth may move — so a restart reproduces the estimator over whatever history was evaluated, not the trajectory the run would have taken (Appendix D). For an efficient implementation we leverage the existing Propagate framework, whose propagator exposes a single call `__call__(inds) -> Individual`: it receives the entire evaluated history and emits the next candidate without persistent archive state or assimilation callbacks, exactly the history-based interface our design calls for. The archive, the bandwidth and the reported weights are therefore reconstructed from history rather than carried. Define the evaluated history at call n as

$$\mathcal{H}_n = \{(\theta_i, \rho_i, \epsilon_i, \tau_i, w_i)\}_{i=1}^n, \quad (3)$$

where θ_i is the parameter, ρ_i the discrepancy, ϵ_i the bandwidth active when θ_i was proposed, τ_i the proposal-time index, and w_i the stored core importance weight π/q_{τ_i} .

3.2 Tolerance reconstruction and monotonicity

Each proposed individual stores the bandwidth active at proposal time. The effective bandwidth reconstructed from history is the running minimum of these stored bandwidths, $\epsilon_{\text{hist}} = \min_i \epsilon_i$ (falling back to the initial tolerance on an empty history). The scheduler proposes ϵ_{sched} , and the bandwidth used is $\epsilon_n = \min(\epsilon_{\text{hist}}, \epsilon_{\text{sched}})$, giving a monotone-decrease guarantee with no mutable propagator state. The schedule starts from ϵ_0 (`tol_init`) and tightens at a bounded rate — each rank’s scheduler searches once per k of its own calls, and a search may at most halve the bandwidth — towards the bandwidth at which $2k$ evaluated particles lie within it; on an expensive simulator, or on any run in which no rank makes many multiples of k calls, that transient outlasts the run, so ϵ_0 is set from the prior-predictive discrepancy scale (§7.4).

3.3 Archive and smooth-kernel proposal

The reconstructed archive is the top- k history members by lowest discrepancy, $A_n = \text{Top}_k(\{\theta_i\}, \text{order by } \rho_i)$. The proposal mixture uses kernel-weighted archive members,

$$q_n(\theta) = \sum_{j \in A_n} \tilde{W}_j^{(n)} K_\Sigma(\theta - \theta_j), \quad \tilde{W}_j^{(n)} \propto w_j \cdot K_{\epsilon_n}(\rho_j), \quad (4)$$

with $\sum_j \tilde{W}_j^{(n)} = 1$. The perturbation kernel K_Σ is Gaussian with covariance $\Sigma_n = s \widehat{\text{Cov}}_{\tilde{W}}(A_n)$ estimated from the kernel-weighted archive (s is the `perturbation_scale` hyperparameter), factored once via Cholesky. Smooth kernels remove the prior-vs-archive discontinuity present in hard-threshold ABC-PMC: every archive member contributes continuously through its kernel weight, though membership of the archive itself still changes at the k -th rank.

3.4 Streaming AMIS importance weight

A particle θ^* drawn from q_n receives an importance weight under the balance heuristic over a sliding ring buffer $\mathcal{S}$ of past proposals (size S , sampled every `amis_interval` calls):

$$w^* = \frac{\pi(\theta^*)}{\bar{q}_n^{\text{on}}(\theta^*)}, \quad \bar{q}_n^{\text{on}}(\theta) = \frac{1}{1 + |\mathcal{S}|} \left[q_n(\theta) + \sum_{s \in \mathcal{S}} q_s(\theta) \right]. \quad (5)$$

Setting $|\mathcal{S}| = 0$ recovers single-current-proposal weighting; increasing S enriches the balance-heuristic denominator toward the full cumulative mixture, which lowers the variance of the importance weights at a modest additional per-call cost. We use $S = 20$ by default, a value that captures most of that variance reduction while keeping each update cheap. A particle’s weight, assigned at proposal time, is corrected by the cumulative-mixture denominator at every subsequent use (the retroactive AMIS step). Each snapshot q_s is the proposal reconstructed from the history truncated at call s ; the running sampler carries the buffer between calls (§3.1), and the reported estimator rebuilds its own snapshots from $\mathcal{H}_n$ (Appendix C). Equation (5) is the proposal-time importance weight; the *posterior* estimator reported in §4 reweights it by the smooth ABC likelihood $K_{\epsilon_n}(\rho)$ (8).

Three weights, one of them reported.

The method uses three weightings and reports one. The *adaptation weights* $\tilde{W}_j^{(n)}$ of (4) are kernel weights over the archive and define the next proposal. The *proposal-time weight* w^* of (5) is stored with each particle and drives parent selection and the online effective-sample-size diagnostic; it never enters the reported posterior. The *reported posterior* is the retroactive estimator $\hat{\pi}_n$ of (8), recomputed from the history on demand: every evaluated particle is reweighted by $\pi(\theta_i)K_{\epsilon_n}(\rho_i)/\bar{q}_n(\theta_i)$, where $\bar{q}_n$ is a draw-proportional mixture of $m \leq S+1$ history-reconstructed proposals with a defensive prior component (Appendix C) — a fixed-size stand-in for the mixture the sample was drawn from, whose cost §4 makes precise. Each stored record holds the fields of (3); the archive, the adaptation weights and the reported posterior are pure functions of them.

3.5 Update loop

Algorithm 1 assembles the pieces above into the single-arrival update the propagator runs on each call. Every quantity it uses (the effective bandwidth, the archive, the proposal mixture, and the AMIS denominator) is reconstructed from the history $\mathcal{H}_n$ passed in, so the reported estimator depends on no state carried between calls (the running sampler’s snapshot buffer and scheduler throttle are the exceptions of §3.1); until the history holds at least k members it emits uniform prior draws to seed the archive.

Algorithm 1 History-reconstructed, single-arrival-driven ABC update

Require: History $\mathcal{H}_n$, archive size k , initial bandwidth ϵ_0 , snapshot buffer $\mathcal{S}$

- 1: Reconstruct ϵ_{hist} from stored tolerances; compute ϵ_{sched} ; set $\epsilon_n \leftarrow \min(\epsilon_{\text{hist}}, \epsilon_{\text{sched}})$
 - 2: **if** $|\mathcal{H}_n| < k$ **then**
 - 3: emit a uniform prior draw (bootstrap) $\triangleright$ covers $\bar{q}$ on the whole box
 - 4: **else**
 - 5: select archive A_n (top- k by ρ); form $\tilde{W}^{(n)}$ in log-space via K_{ϵ_n}
 - 6: build Σ_n , Cholesky-factor once; sample θ^* by perturbing a $\tilde{W}^{(n)}$ -weighted parent
 - 7: compute w^* via the AMIS denominator (5); periodically snapshot q_n into $\mathcal{S}$
 - 8: **end if**
 - 9: **return** θ^* for external simulation and discrepancy evaluation
-

3.6 Relation to classical ABC-SMC

Table 1 sets the method against the generation-staged ABC-SMC/PMC implementation we compare with — the pyABC baseline — property by property. None of the listed properties is intrinsic to ABC-SMC as a family (smooth kernels and richer proposals exist), but each characterises the configured baseline. The decisive row is the last: because all state is a pure function of the history, there is nothing to synchronise between workers, which is what fits the method to the asynchronous island model of §5.

4 Theoretical Analysis

We prove consistency, up to a fidelity ratio, for the estimator this paper reports, as computed by a post-hoc pass over the log the algorithm writes; the proofs index particles by proposal time and the pass by arrival time, and Appendix B states what closing that gap assumes. The proof is self-contained rather than a corollary of AMIS theory — the published consistency results for recycling schemes [14] restrict the adaptation so that the weighting stage decouples, which a single-arrival scheme does not do — and it carries the implementation’s departures from the ideal inside the statement, as a single ratio r , rather than beside it as a list of caveats. A central limit theorem for damped-adaptation variants, with a per-run variance estimator, and

Table 1 A typical generation-staged ABC-SMC/PMC implementation (the pyABC baseline used here) versus the proposed generation-free method. The listed properties characterize the configured baseline, not ABC-SMC as a family. The running sampler does carry the online snapshot buffer of §3.4; the reported estimator rebuilds it from the history.

Property	Classical ABC-SMC	This work
Update style	generation, barrier-synchronized	event-driven (single-arrival), no barrier
Archive	explicit, generational	reconstructed, sliding top- k
ABC likelihood	hard threshold	smooth kernel
Importance weight	against q_{t-1} only	balance heuristic over $\mathcal{S}$
Archive state	mutable, carried	none; reconstructed per call
Reported estimator	final population	pure function of the history

the decomposition and partial measurement of r are in Appendix A; the proofs in Appendix B.

The analysis is organized around a single question, because that is all the proofs turn out to need: *is the denominator the estimator divides by the density its sample was actually drawn from?* Write $\tilde{q}_i$ for the conditional density of the i -th candidate given the past, in the sense of the filtration fixed by Assumption 4. It absorbs every internal departure of the implementation — the top- k archive rule, the capped in-box rejection, the underflow redraws, the bootstrap phase — and it is the *only* way the sampler enters Theorem 1. That such a conditional density exists with respect to a filtration the proofs can use is itself an assumption once execution is asynchronous, which is why Assumption 4 states it rather than leaving it implicit. Sequentially it is exactly the density the propagator emits from; asynchronously the two can differ, and Assumption 4 is where that is confronted rather than hidden. Write

$$\bar{q}_n^\star := \frac{1}{n} \sum_{i \leq n} \tilde{q}_i \quad (6)$$

for the mixture the pooled sample was drawn from, against the reported denominator

$$\bar{q}_n = w_n \pi + (1 - w_n) \sum_{s=1}^m \alpha_{s,n} q_{\tau_s}, \quad w_n = \max(\nu_n, \delta), \quad \alpha_{s,n} \geq 0, \quad \sum_s \alpha_{s,n} = 1, \quad (7)$$

the draw-proportional deterministic mixture [13, 24] of $m \leq S+1 = 21$ history-reconstructed proposals with a prior component whose mass is the observed bootstrap share ν_n floored at $\delta = 0.5/(m+1)$, exactly as the implementation builds it (Appendix C). Throughout, ϵ_∞ denotes the limit of the bandwidth schedule $\{\epsilon_n\}$, whose existence and positivity are part of Assumption 5(i); $L_\epsilon(\theta) = \mathbb{E}[K_\epsilon(\rho) \mid \theta]$ is the smooth ABC likelihood (1), $L_\epsilon^{(2)}(\theta) = \mathbb{E}[K_\epsilon(\rho)^2 \mid \theta]$, and the estimator is

$$\hat{\pi}_n(f) = \frac{\sum_{i=1}^n W_{i,n} f(\theta_i)}{\sum_{i=1}^n W_{i,n}}, \quad W_{i,n} = \frac{\pi(\theta_i) K_{\epsilon_n}(\rho_i)}{\bar{q}_n(\theta_i)}. \quad (8)$$

Assumptions.

Assumption 1 (kernel). $K_\epsilon : [0, \infty) \rightarrow [0, 1]$ is measurable with $K_\epsilon(0) = 1$ (unit peak, not unit integral, as is standard for a probabilistic-acceptance ABC kernel), and for every $a \in (0, \epsilon_0]$ there is $L_K(a) < \infty$ with $|K_\epsilon(\rho) - K_{\epsilon'}(\rho)| \leq L_K(a)|\epsilon - \epsilon'|$ for all $\rho \geq 0$ and $\epsilon, \epsilon' \in [a, \epsilon_0]$. The limiting target is non-degenerate: $Z_{\epsilon_\infty} = \int_\Theta \pi L_{\epsilon_\infty} > 0$.

Assumption 2 (denominator class). $\Theta \subset \mathbb{R}^d$ is a non-degenerate axis-aligned box and the prior density satisfies $0 < \pi_- \leq \pi \leq \pi_+ < \infty$ on it. Each q_{τ_s} in (7) is a positive function on Θ of truncated-Gaussian-mixture form — at most k components, means in Θ , weights in the simplex, covariance Σ with $\sigma_-^2 I \preceq \Sigma \preceq \sigma_+^2 I$ for fixed $0 < \sigma_- \leq \sigma_+ < \infty$, and per-component normalizing constants evaluated as the implementation evaluates them. Nothing requires $\bar{q}_n$ to integrate to one: the proofs use only positivity, boundedness and Lipschitz dependence on the parameters, so an approximate normalizing constant leaves Assumption 2 intact and shows up instead as a contribution to r .

Assumption 3 (prior floor in the denominator). $\delta \in (0, 1/2]$ is fixed and $w_n \geq \delta$ in (7).

Assumption 4 (adapted filtration). There is a filtration $(\mathcal{F}_i)$ with $\xi_i = (\theta_i, \rho_i)$ $\mathcal{F}_i$ -measurable, $\theta_i \mid \mathcal{F}_{i-1} \sim \tilde{q}_i$ and $\rho_i \mid \mathcal{F}_{i-1} \vee \sigma(\theta_i) \sim p(\cdot \mid \theta_i)$. Under sequential execution this holds by construction; under asynchronous execution it is a genuine assumption (Appendix B).

Assumption 5 (fidelity and stabilization). (*i, consistency*) $\epsilon_n \downarrow \epsilon_\infty > 0$ almost surely (ϵ_∞ may be random), and the denominator tracks the draw mixture up to a limiting ratio: for some r with $0 < r_- \leq r \leq r_+ < \infty$,

$$\|\bar{q}_n^*/\bar{q}_n - r\|_\infty \longrightarrow 0 \quad \text{a.s.} \quad (9)$$

(*ii, central limit theorem*) In addition ϵ_∞ and r are deterministic, a (possibly random) limit $\tilde{q}_\infty$ of $\tilde{q}_i$ and $\bar{q}_\infty$ of $\bar{q}_n$ exist with $r = \tilde{q}_\infty/\bar{q}_\infty$, and, almost surely,

$$\begin{aligned} \sqrt{n} (|\epsilon_n - \epsilon_\infty| + \|\bar{q}_n - \bar{q}_\infty\|_\infty) &\rightarrow 0, \\ \frac{1}{\sqrt{n}} \sum_{i \leq n} (|\epsilon_{i-1} - \epsilon_\infty| + \|\tilde{q}_i - \tilde{q}_\infty\|_\infty + \|\bar{q}_{i-1} - \bar{q}_\infty\|_\infty) &\rightarrow 0, \end{aligned} \quad (10)$$

where ϵ_{i-1} and $\bar{q}_{i-1}$ are the bandwidth and the denominator (7) formed from the first $i - 1$ draws (both known before draw i ; convention $\bar{q}_0 := \pi$).

Assumption 1 holds for the Gaussian kernel with $L_K(a) = 2/(ea)$ and Epanechnikov with $2/a$; it excludes the hard indicator, which is not Lipschitz in ϵ — one concrete reason the method uses smooth kernels. The Lipschitz constant is stated per compact sub-interval because it degrades as $\epsilon \downarrow 0$; no *deterministic* bandwidth floor is needed, since the proof localizes over $[1/j, \epsilon_0]$ and intersects countably many almost-sure events (Appendix B). Assumptions 2 and 3 constrain only the *reported*

denominator, which the implementation builds to that exact recipe. Assumption 5 carries everything else. For *consistency*, (9) is the whole of the sampler’s contribution: not that the sampler take any particular form, but that whatever it does be asymptotically reflected in the denominator, up to a ratio r the results carry rather than assume away. The central limit theorem asks more — the per-draw rate in (10) and, through the variance, the limit $\tilde{q}_\infty$ itself — so the “one channel” reading is exact for Theorem 1 and approximate for Theorem 3. Note the two clauses differ in strength: consistency permits r and ϵ_∞ to be random, while the central limit theorem needs them deterministic, because the centred integrand must be a fixed function for the martingale arrays to be adapted. That restriction costs nothing in the case of most interest, $r \equiv 1$, which is deterministic no matter how run-dependent $\tilde{q}_\infty$ is.

Theorem 1 (consistency). *Let Assumptions 1–4 and 5(i) hold. Then for every bounded measurable $f : \Theta \rightarrow \mathbb{R}$,*

$$\hat{\pi}_n(f) \longrightarrow \pi_{\epsilon_\infty}^r(f) := \frac{\int_{\Theta} f \pi L_{\epsilon_\infty} r}{\int_{\Theta} \pi L_{\epsilon_\infty} r} \quad \text{almost surely.} \quad (11)$$

Corollary 2 (exact fidelity). *If $r \equiv 1$ — the denominator is asymptotically the density the sample was drawn from — then $\hat{\pi}_n(f) \rightarrow \pi_{\epsilon_\infty}(f)$, the smooth-ABC posterior at the limiting bandwidth.*

What r is, and how much of it is measured.

Everything specific to the code enters through r : the prior floor in the denominator, the m -point snapshot quadrature standing in for the n -point mixture, the product-of-marginals truncation constant, and the capped in-box rejection and underflow redraws (which under asynchronous execution also absorb the gap between the emitted proposal density and the conditional law of Assumption 4). The first two can be estimated from a stored run by recomputing the denominator at large m and the observed prior share: on a Cellular Potts history (the benchmark’s earlier configuration, Appendix E) the reported posterior lies 1.1% in total variation from that reference, and on a one-dimensional Gaussian history 0.26%. The other two are bounded, not measured, so this is a partial accounting of r and the results are conditional on its finite positive limit (Appendix A). The reported runs also leave the bandwidth floor unset, so the positive limit $\epsilon_\infty > 0$ is assumed rather than enforced for them (Table B1). The central limit theorem asks for a stabilisation rate that the reported configuration measurably misses; the reason is the inference problem, not the sampler — a weakly identified direction lets the archive slide indefinitely — and an identified target meets the rate with room to spare.

5 Implementation in Propulate

The algorithm is a Propulate propagator that carries no archive between calls: tolerance schedulers are pure functions of history; the archive is the top- k history members

by smallest discrepancy under the reconstructed bandwidth; the smooth-kernel mixture weights are formed in log-space for numerical stability. The reported posterior is the retroactive AMIS estimator (8), computed off the timed inference path from the saved history (a pure function of it, hence identical whether computed online or after a crash/restart); the bare proposal-time weight (5) is retained separately for the effective-sample-size diagnostic. This retroactive step is a one-time pass of cost $\mathcal{O}(nk)$ in the n evaluated particles and archive size k , run after the timed budget and therefore excluded from the throughput measurements by design (the systems comparison is over simulation throughput). For cost-bearing simulators it is negligible against the simulation time; for near-instantaneous simulators whose history reaches $\mathcal{O}(10^7)$ particles it is non-trivial but bounded ($\mathcal{O}(nk)$ with the fixed $m \leq S+1 = 21$ snapshot count), and is evaluated in fixed-memory chunks (Appendix C) so it stays a bounded fraction of wall-clock and memory. For an apples-to-apples comparison, pyABC’s **UniformAcceptor** is replaced by a probabilistic-rejection acceptor using the same $K_\epsilon(\rho)$ and bandwidth schedule as the asynchronous side. The acceptor and kernel are thus matched across methods; the synchronous baseline differs in its generation-staged *execution* (the systems variable under test) and, on the inference side, in retaining classical population weighting rather than the streaming AMIS estimator, whose quality we check separately (§7.4). Two further instruments are built from the same code. The *barrierized twin* subclasses the propagator and inserts a collective barrier over the worker communicator immediately before each breed, so that every rank waits for the slowest before proposing; nothing else — proposal, kernel, archive rule, reported estimator — changes, and in particular histories are not truncated to generation boundaries: each rank proposes from the history it has received at the barrier. The twin therefore isolates the barrier’s timing, not the information structure of a generation. Because a collective needs identical call counts on every rank, the twin runs to a fixed simulation count rather than a wall clock. *Rejection ABC* runs in a best- k mode: it spends the configured budget and reports the k smallest discrepancies, which is the tolerance an adaptive sampler is compared against in §7; a fixed threshold at the shared ϵ_0 would, on an expensive simulator, accept every draw. Distributed execution uses Propulate’s barrier-free island model over MPI; for wall-time-limited HPC experiments the synchronous baseline uses fixed populations/generations, which yields more interpretable comparisons than contrasting methods that stop under different ϵ rules.

6 Experimental Design

6.1 What is tested

The results are organised around four claims. **(C1)** A generation barrier costs the straggler factor of the workload, and that cost is predictable from the asynchronous run’s timing alone. **(C2)** Removing it converts into a tighter tolerance, because the asynchronous sampler is at least as efficient per simulation as the synchronous baseline. **(C3)** The advantage has a boundary in the cost of a simulation, below which per-arrival coordination costs more than the barrier. **(C4)** The reported posterior is calibrated, recovers a known truth on a real simulator, and is limited by two settings we identify. Each claim gets one subsection of §7, and each needs a different kind of

evidence, which is why the benchmark suite spans four orders of magnitude in the cost of a simulation.

6.2 Benchmarks

Table 2 lists the five configurations. The two cheap benchmarks — a one-dimensional Gaussian mean with an analytic posterior and the four-dimensional g-and-k distribution with a reference posterior — are where calibration and recovery can be scored exactly, and where the cost of coordinating a near-instant simulator is exposed; the Gaussian mean also hosts the two studies with *injected* runtime heterogeneity, a persistently slow worker and a per-evaluation lognormal runtime multiplier, in which the runtime distribution is known by construction. Lotka–Volterra is a stochastic simulator whose cost varies with the trajectory and whose parameters send 98% of simulations to extinction (Appendix D); it serves the scaling and boundary measurements and is not scored for posterior quality. The Cellular Potts benchmark runs the real `cellsInSilico` tissue simulator [25] end to end. Its inference problem was designed on a screening campaign (Appendix E): two parameters, the cell division rate and the target cell volume, log-uniform on $[0.001, 0.2]$ and $[200, 1200]$, that move the two directions the summary statistics resolve — the cell count and the cluster radius at fixed count — and a discrepancy over those two statistics averaged over four replicate simulations per evaluation. We run it at two sizes: a 50^3 box, where one evaluation costs 13s, and an 80^3 box with twice the duration, where it costs 170s and the runtime coefficient of variation rises from 0.05 to 0.13–0.23 because runtime tracks cell count ($r = 0.99$) and cell count is no longer bounded by the box.

The Cellular Potts benchmark is a *calibration instrument*, and we state its role so that it is not read as a production run. Production simulations of this model take hours on many cores; ours take seconds to minutes, and their inference is on synthetic data with a known truth. What that buys is the ability to say whether the method recovers a posterior it should recover, and to measure the barrier’s cost on a real workload at two levels of heterogeneity; what a production project takes from it is the predictor of §6.4, which needs only the production run’s own timing.

6.3 Baselines and instruments

Three comparators appear, each for a different question. The *matched synchronous baseline* is pyABC [20, 21] with its acceptor replaced by a probabilistic-rejection acceptor using the same smooth kernel and the same bandwidth schedule as the asynchronous side (§5); it is the method a practitioner would run, and the comparator for C2 and C3. Its population of 100 on 47 simulating ranks (rank 0 dispatches) forces at least three rounds per generation with the last one 13% full, so it idles by construction even on a homogeneous simulator; the scaling study gives it a population equal to the worker count, the production comparisons do not (§9). The *barrierized twin* is our own propagator with a collective barrier inserted before each proposal and nothing else changed (§5); it isolates the timing of synchronization from every other difference and is the comparator for C1. A collective barrier needs identical call counts on every rank, so the twin runs to a fixed simulation count and the asynchronous arm to a

Table 2 Benchmark suite. Cost is the mean wall-clock time of one simulation on the asynchronous arm (an evaluation on Cellular Potts is four replicate simulations); runtime spread is the in-run coefficient of variation of that time, or how it is injected. Budgets are per replicate; five replicates throughout except three at 80³.

Benchmark	d	cost/sim	runtime spread	budget, workers	role
Gaussian mean	1	0.1 ms	injected: straggler 0–20×; lognormal $\sigma \leq 2$	300 s, 48 (16 in the straggler study)	calibration (SBC); barrier cost under a known runtime law; the cheap end of the cost boundary
g-and-k	4	2 ms	none	600 s, 48	4-D calibration and recovery against a reference posterior
Lotka–Volterra	4	4 ms	trajectory-dependent; 98% extinct	600 s, 48; scaling to 288	strong scaling; the cost boundary
Cellular Potts 50 ³	2	13 s	CV 0.05	3600 s, 48; scaling to 384	posterior recovery against a known truth; barrier cost on a real simulator
Cellular Potts 80 ³	2	170 s	CV 0.13–0.23	3600 s, 48	barrier cost at realistic cost and heterogeneity

fixed wall clock, and the two are compared as rates; for the posterior comparison the asynchronous arm is additionally re-run at exactly the twin’s simulation count and seeds. *Rejection ABC* spends the same simulation budget and keeps the k best draws — not a fixed threshold, which on an expensive simulator admits every draw — and is given the whole machine (all 48 ranks) rather than one rank per replicate. It is the reference for what an adaptive proposal buys per simulation.

6.4 Metrics

Tolerance at a matched budget.

The quantity that compares samplers on one footing is the k -th order statistic of a method’s own discrepancies after n simulations, $\epsilon_{(k)}(n)$: the tolerance at which exactly k of its draws would be accepted. It needs no reference posterior, it does not depend on the bandwidth a schedule happened to report at, and it is what rejection ABC reports at by construction. Read at equal n it measures per-simulation efficiency; read at the end of equal wall-clock budgets it measures the net effect; its slope in $\log n$ says whether more simulations keep converting into a tighter tolerance. We use $k = 100$ throughout, count only records of completed simulations (the synchronous baseline also writes one record per accepted particle, and mixing the two flattens it), and order records by completion time.

Throughput, utilisation, and the predicted barrier cost.

Throughput is simulations completed per second of wall clock; utilisation is the fraction of worker time spent inside the simulator. The prediction that C1 tests is built from the asynchronous run only: with D_i the per-evaluation durations on that run — recorded simulation time plus the measured per-evaluation coordination overhead — a barrierized generation of W evaluations takes $\mathbb{E}[\max_{i \leq W} D_i]$, so the synchronous

throughput is predicted as $T_{\text{sync}} = W/\mathbb{E}[\max_{i \leq W} D_i]$, the expectation taken by resampling W durations from the asynchronous record (or from the injected law, where the heterogeneity is injected; a persistently slow worker contributes its own mean). The predicted cost is $T_{\text{async}}/T_{\text{sync}}$ with T_{async} the measured asynchronous throughput, and it is compared with the ratio measured against the twin. For identically distributed, uncensored durations $T_{\text{async}} = W/\mathbb{E}[D]$ and the ratio is the straggler factor of §1; with a persistently slow worker, or when a wall-clock budget censors the long durations, the two differ, and the general form is the one we use.

Posterior quality.

Calibration is checked by simulation-based calibration (SBC): empirical coverage of the reported posterior’s central intervals over 1000 trials at the 0.50/0.80/0.90/0.95 levels. Where a reference posterior exists we report the Wasserstein distance W_1 between each method’s *own reported* posterior and that reference (in more than one dimension the sliced W_1 over random projections, on the parameters’ natural scale) — the analytic posterior for the Gaussian mean, and for g-and-k the posterior given the same octile summaries the discrepancy is built from, obtained from their asymptotic law (Appendix D); the asynchronous estimator is replayed over the prefix of the history completed by each checkpoint, the synchronous baseline is scored on its last completed generation with pyABC’s population weights where the history was kept and uniformly otherwise, and rejection ABC on its k best draws. On Cellular Potts, whose truth is known but whose posterior is not, we report each parameter’s *contraction* $1 - \text{sd}/\text{sd}_{\text{prior}}$ in prior-normalised units, whether the central 90% interval covers the truth, and the effective sample size of the reported weights. We no longer report the distance from an unweighted archive to a point mass at the truth: that statistic is floored at the posterior’s own spread and rewards a collapsed posterior over a correct one (Appendix F).

6.5 Studies

C1 is tested on the straggler study (one of 16 workers given a permanent post-evaluation delay of 0.1 s scaled by 0, 1, 5, 10, 20), the heterogeneity study (every evaluation on 48 workers multiplied by a lognormal factor of scale $\sigma \in \{0, 0.5, 1, 1.5, 2\}$), and the two Cellular Potts sizes, at 48 to 384 workers. C2 and C3 use the production runs of all four benchmarks at 48 workers and fixed wall clock (12 ranks per node on four nodes for the three cheap benchmarks, one node for Cellular Potts; Appendix D). C4 uses SBC on the Gaussian mean, a bimodal variant of it, and g-and-k, the reference-posterior recovery on the two benchmarks that admit one, and the Cellular Potts 50³ production runs, with and without the bandwidth defect described in §7.4. Strong scaling to six nodes on Lotka–Volterra and eight on Cellular Potts enters C1 and C3; its tables and timing decomposition, the hyperparameter sensitivity grid, the ablation of the kernel and of the AMIS reweighting, the calibration sweep over the archive size, and the parameter-coupled-runtime study are reported in the supplement. All experiments ran on JUWELS (Appendix D).

7 Results

Unless noted, results are medians over five replicates with the replicate range shown; *asynchronous* is the proposed method, *synchronous baseline* the matched-kernel pyABC comparator, and *twin* the barrierized copy of our own propagator (§6.3). The four subsections follow the four claims of §6.1.

7.1 The barrier costs the straggler factor, and the cost is predictable (C1)

Figure 1 is the paper’s central measurement. For every configuration on which we ran the barrierized twin — a persistently slow worker at five slowdowns, lognormal runtime heterogeneity at five spreads, the Cellular Potts simulator at four worker counts and two sizes — it plots the barrier’s cost as *predicted* from the asynchronous run’s timing alone (§6.4) against the cost *measured* against the twin. Twelve configurations lie inside the model’s stated domain; over them the predicted-to-measured ratio runs from 0.84 to 1.19 with median 0.996, across measured costs from $1.2\times$ to $400\times$. No synchronous or twin data enter the prediction. Three configurations lie outside the domain and are drawn open: the cost-free simulator at $0\times$ and $1\times$, where the collective’s own latency dominates and the model has no latency term, and 384 Cellular Potts ranks, where the asynchronous timing sample carries a contention tail the twin’s generations did not show. The rest of this subsection walks through the three workloads.

A persistent straggler.

One of 16 workers is given a permanent post-evaluation delay of 0.1s scaled by 0, 1, 5, 10, 20 (Fig. 2, Table 3). The asynchronous arm, wall-limited, holds 3200–3900 simulations per second at every slowdown; the twin, which must run to a fixed simulation count, falls from 48.7 to 8.0 with a barrier after every $W = 16$ evaluations and from 163 to 8.0 with a barrier after every 112 (the baseline’s population); the synchronous pyABC baseline falls from ≈ 8800 to ≈ 60 . The prediction is a one-line calculation: the slow worker’s recorded evaluation time (2.000s at $20\times$) plus the 3.6ms per-evaluation overhead measured on the fast workers is the generation time, so the twin cannot exceed $16/2.004 = 7.98$ simulations per second, and the measured asynchronous rate divided by that gives $401\times$ against a measured $402\times$ (this is the throughput ratio of §6.4, not the straggler factor of the sixteen workers’ runtimes, which would be 15.6: on the asynchronous arm the fifteen fast workers never wait); at $5\times$ and $10\times$ the figures are 102 against 103 and 202 against 202. At $0\times$ and $1\times$ the prediction fails, and the failure is instructive: with no straggler to wait for, the collective itself costs about 0.3s per barrier on a 4ms workload, which the model does not contain. Coarsening the barrier to one per 112 evaluations amortises that latency and buys $3.3\times$ at the control, and buys nothing at all from $5\times$ up, where the three granularities agree to three significant figures: once a straggler binds, the barrier’s cost does not depend on how often it is taken. Two details of the prediction generalise. The slow worker’s cost is its recorded *evaluation* time, not its wall-clock span on the asynchronous run, which also includes ingesting the ≈ 3000 arrivals per second the other ranks produce — a cost asynchrony alone pays. And the asynchronous arm’s timing record must not

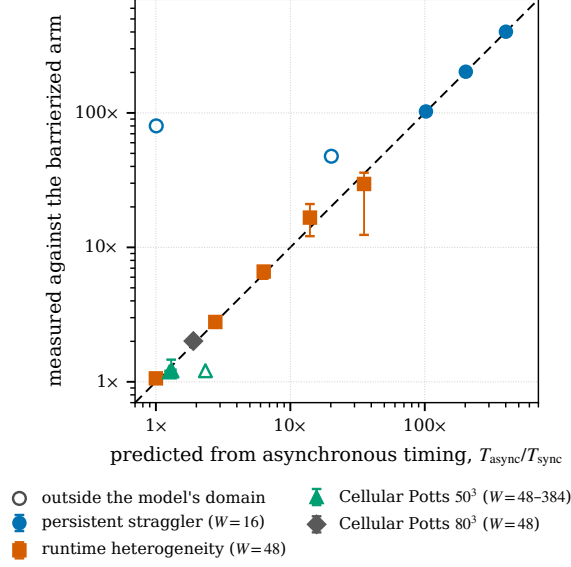


Fig. 1 The barrier’s cost, predicted from the asynchronous arm’s timing alone (§6.4) against the cost measured with the barrierized twin, on every workload: a persistent straggler on 16 workers (0–20×), lognormal runtime heterogeneity on 48 workers ($\sigma = 0$ –2), Cellular Potts at 50^3 (48–384 workers, measured as the utilisation ratio; the benchmark’s earlier single-simulation configuration, Appendix E) and at 80^3 (48 workers, against the matched pyABC baseline rather than the twin). Points are medians over replicates with the replicate range; the dashed line is equality. Open markers are the three configurations outside the model’s domain: the 4 ms simulator at 0× and 1×, where barrier latency rather than waiting dominates, and 384 Cellular Potts ranks, where the asynchronous sample carries a contention tail.

be censored by its own budget: here the delays are known by construction, but under the lognormal heterogeneity below the 60s budget truncates the empirical tail and a prediction from the truncated sample undershoots by 2–4× at the two largest spreads.

Runtime heterogeneity.

Every evaluation on 48 workers is multiplied by a lognormal factor of scale σ (Table 4), so the slow draw is a different worker each time. The control costs almost nothing (1.06×, predicted 1.00): these evaluations take 1 s, and against that the collective’s latency is negligible, so the barrier’s cost is a pure waiting cost and appears only once there is variance to wait for. It then grows to 2.8, 6.6, 16.6 and 29.6×, against 2.77, 6.34, 14.0 and 35.4× predicted from the injected law — misses of 16% and 19% at the two largest spreads, the largest in Fig. 1. $\mathbb{E}[\max_{48}]$ of a lognormal of scale 2 is 147 s against a mean of 7.4, so the straggler factor proper is 19.9; the predicted and measured ratios are larger because the 60 s budget censors the asynchronous arm’s longest draws, lifting its throughput to 4.2 s per evaluation per worker against the law’s 7.4, and the prediction inherits that censoring through T_{async} as the measurement contains it. The expectation is set by rare draws — each twin replicate at $\sigma = 2$ has only ten generations and one of them drew a 993 s evaluation — so at the two

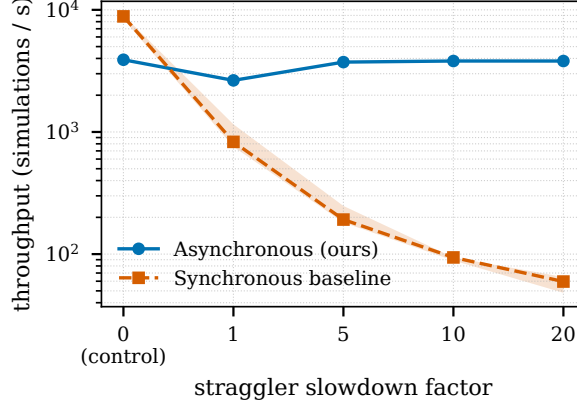


Fig. 2 Throughput (median and inter-quartile range over five replicates) versus straggler slowdown on the Gaussian-mean benchmark, including the $0\times$ control. One of 16 workers carries a permanent post-evaluation delay of 0.1 s scaled by the factor. The synchronous baseline falls from ≈ 8800 to ≈ 60 simulations per second because every generation waits for it; the asynchronous method holds ≈ 2600 – 3900 . This is the 300 s wall-limited campaign; the twin campaign’s asynchronous arm in Table 3 agrees within 15% except at $1\times$ (2645 here against 3112 there). Logarithmic throughput axis.

largest spreads it is the measurement that is noisy, not the model. Here coarsening the barrier does help, by 1.2 – $1.7\times$, because a transient slow draw is amortised over a longer generation where a persistent one is not. The pyABC baseline degrades far less than our own twin, from 2.1 to $4.8\times$, because it closes a generation on the first 100 finishers and discards the latecomers: the headline comparison against it therefore understates the barrier’s unmitigated cost, which the twin measures.

The real simulator.

On Cellular Potts at 50^3 the twin ran at 48, 96, 192 and 384 workers — on the benchmark’s earlier single-simulation configuration (5.1 s per evaluation, division rate and motility; Appendix E), not the production setup of Table 2 — and its throughput was $1.9\times$ below the asynchronous arm’s at the first three and $2.8\times$ at the last (Table 5). That factor is not the barrier’s. Throughput is utilisation over mean simulation time, and the two factors separate exactly: the utilisation ratio is 1.18 – $1.21\times$ at every worker count — the asynchronous arm at 99–99.8%, the twin at 82–85% — and equals the straggler factor of the twin’s own generations (1.21 – 1.29) and the prediction from the asynchronous arm’s runtime distribution (coefficient of variation 0.10 on those runs; 1.26 – $1.32\times$ at 48–192 workers). The remaining 1.5 – $2.3\times$ is the twin’s simulations taking longer — 6 to 36 s across replicates against 5.1 s on every asynchronous replicate at every worker count, with no correlation to the parameters — which a barrier cannot cause. Synchronised I/O bursts when W simulations start together, or filesystem load at the time of those runs, are the candidates; the first would be a secondary cost of the barrier, the second none of its doing. We do not attribute it, and we do not count it, so the utilisation ratio we quote as the barrier’s cost on this workload is a lower bound. What the barrier costs on this workload is $1.2\times$, on the ceiling its runtime spread sets.

Table 3 Barrierized twin on the straggler benchmark ($W = 16$, five replicates, medians). Throughput is simulations per second of active wall clock; the asynchronous arm is wall-limited, the twin runs to a fixed simulation count. *Predicted* is $W/\mathbb{E}[\max D]$ from the asynchronous run’s timing alone (§6.4); at $0\times$ and $1\times$ the collective’s latency, absent from the model, dominates. The lower block scores each arm’s *reported* posterior against the analytic posterior (W_1) at the bandwidth its schedule reached, listed against the 100th-smallest discrepancy of the same history; the asynchronous rows are the method re-run at exactly the twin’s simulation count and seeds, so the arms differ only in whether workers wait. The last row re-reports the twin’s histories at $\epsilon_{(100)}$ (§7.4); the asynchronous arm re-reported the same way gives 0.010–0.015. Recomputed from the stored records by one code path (`make_twin_rereport.py`).

Straggler slowdown	$0\times$	$1\times$	$5\times$	$10\times$	$20\times$
Asynchronous, sims/s	3888	3112	3241	3218	3205
Twin, barrier every W	48.7	65.3	31.6	15.9	8.0
Twin, barrier every 112	162.8	148.4	31.8	15.9	8.0
Twin, predicted	(3870)	(154)	31.8	15.9	7.98
Ratio, measured (every W)	$80\times$	$48\times$	$103\times$	$202\times$	$402\times$
Ratio, predicted	—	—	$102\times$	$202\times$	$401\times$
<i>Reported posterior, W_1 to the analytic posterior, at the twin’s simulation count</i>					
Evaluations (= calls per rank $\times 16$)	75,104	44,512	12,304	6,304	3,200
Bandwidth reached, asynchronous	5×10^{-4}	7×10^{-4}	0.003	0.006	0.16
Bandwidth reached, twin every W	5×10^{-4}	8×10^{-4}	0.06	1.3	2.9
$\epsilon_{(100)}$ of the same history	3×10^{-4}	5×10^{-4}	0.002	0.003	0.008
Asynchronous at that count	0.011	0.009	0.010	0.010	0.068
Twin, barrier every W	0.013	0.007	0.016	0.789	1.713
Twin, barrier every 112	0.009	0.009	0.009	0.017	1.082
Twin every W , re-reported at $\epsilon_{(100)}$	0.015	0.010	0.010	0.010	0.010

That ceiling rises with the simulator’s size, and the 80^3 configuration tests the prediction where it matters. There each evaluation costs 170s, the in-run runtime coefficient of variation is 0.23 on the synchronous arm (0.13 on the asynchronous one, which concentrates as it runs; the prediction uses the arm’s own in-run distribution, not a prior-wide screen), and against the matched pyABC baseline the asynchronous arm’s utilisation is 98% against 49% and its throughput $2.1\times$ — predicted $1.90\times$. At 50^3 the same comparison against pyABC gives $2.3\times$ on the scaling campaign and $2.4\times$ on the production run, of which only $1.2\times$ is the barrier: the rest is pyABC’s fixed per-generation overhead, which is a large fraction of a 13s evaluation and negligible against a 170s one. We therefore quote $2.0\times$ for the Cellular Potts systems gain, because it is the number attributable to the mechanism the paper is about. The asynchronous method itself scales almost linearly on this workload, at 97% parallel efficiency from one to eight nodes (Fig. 3, right). At the 80^3 runtime spread the straggler factor at 384 workers is a predicted $2.1\times$; the larger gap against pyABC visible at that scale at 50^3 is, again, overhead.

What the barrier does to the reported bandwidth.

The twin’s throughput is not the only thing that changes under a barrier. Scored on the estimator this paper reports, at the bandwidth its schedule reached, the twin’s

Table 4 Barrierized twin under runtime heterogeneity ($W = 48$, lognormal runtime multiplier of scale σ , five replicates, medians of simulations per second of each arm’s own elapsed wall clock). *Predicted* is from the injected law and the asynchronous arm’s measured overhead. The $\sigma = 0$ control costs almost nothing because these evaluations take 1 s and barrier latency is negligible against them; the cost is the wait for the slowest draw and grows with σ . The pyABC baseline degrades less than the twin because it discards latecomers instead of waiting for them. The lower block scores the reported posteriors against the analytic posterior at the bandwidth each schedule reached, then re-reported at the 100th order statistic of the same history (§7.1); no rank made k calls on the asynchronous arm, so its schedule never ran at $\sigma \leq 0.5$, and the twin’s ran only at $\sigma \leq 0.5$.

Heterogeneity σ	0	0.5	1.0	1.5	2.0
Asynchronous, sims/s	47.6	42.1	28.9	17.6	11.4
Twin, barrier every W	44.8	15.1	4.37	1.06	0.37
Twin, barrier every 96	45.1	19.8	6.50	1.75	0.45
Synchronous pyABC baseline	23.1	18.9	9.91	4.35	2.39
Ratio, measured (every W)	1.06×	2.8×	6.6×	16.6×	29.6×
Ratio, predicted	1.00×	2.77×	6.34×	14.0×	35.4×
Ratio, pyABC baseline	2.1×	2.2×	2.9×	4.1×	4.8×
<i>Reported posterior, W_1 to the analytic posterior (calls per rank: async 59–13, twin 440–10)</i>					
Evaluations, async	2832	2527	1731	1055	684
Evaluations, twin	21,120	6,912	2,112	672	480
Bandwidth reached, async	5	5	0.92	1.25	0.78
Bandwidth reached, twin every W	0.36	2.4	5	5	5
Async, at the schedule’s bandwidth	2.12	2.04	0.68	0.93	0.56
Twin every W , at the schedule’s bandwidth	0.19	1.43	2.20	2.22	2.28
pyABC baseline, last generation	0.53	0.61	1.36	2.19	2.21
Async, re-reported at $\epsilon_{(100)}$	0.011	0.011	0.010	0.013	0.042
Twin every W , re-reported at $\epsilon_{(100)}$	0.015	0.013	0.012	0.040	0.203
Async archive (top 100)	0.018	0.017	0.016	0.014	0.019

posterior collapses once the straggler binds — $W_1 = 0.79$ at $10\times$ and 1.71 at $20\times$ with a barrier every W (Table 3, lower block) — while the asynchronous method re-run at exactly the twin’s simulation count and seeds reports 0.010 and 0.068. Same propagator, same seeds, same number of evaluations, and the same 200 calls per rank at $20\times$; what differs is the bandwidth the schedule reached, 2.9 on the twin against 0.16 on the asynchronous arm (medians), on histories whose 100th-smallest discrepancy is 0.008. The reason is in the scheduler, not in the estimator. Its bandwidth search is throttled to once per k calls on each rank, and is otherwise re-run only when a rank rebuilds its cached history after an out-of-order arrival (Appendix C); asynchronous arrivals trigger those rebuilds continually, lockstepped ones never do, so on a short history the twin’s bandwidth stops at its throttle points and the reported posterior is close to the prior-weighted history. The estimator itself is unharmed: re-reporting the same twin histories at the 100th order statistic of their own losses — the check of §7.4 — brings them to the asynchronous arm’s level (Table 3, last rows). The heterogeneity campaign shows the same effect on *both* arms, because its 60 s budget lets no rank

Table 5 The twin’s slowdown on Cellular Potts 50^3 in the benchmark’s earlier single-simulation configuration (Appendix E), factorised (five replicates, medians). Throughput is utilisation over mean simulation time, so the ratio is the product of the two columns that follow. The utilisation ratio is what the barrier costs; it agrees with the straggler factor measured inside the twin’s own generations and with the prediction from the asynchronous arm’s runtime distribution. The duration ratio — the twin’s simulations running longer, with replicate means from 6 to 36 s against 5.1 s on every asynchronous replicate — is not a barrier effect and is not attributed here.

Workers (= cores)	48	96	192	384
Throughput ratio, async / twin	1.91×	1.87×	1.86×	2.82×
= utilisation ratio	1.18×	1.19×	1.21×	1.21×
× simulation-duration ratio	1.63×	1.59×	1.52×	2.33×
Straggler factor of the twin’s generations	1.21	1.28	1.21	1.29
Predicted from asynchronous timing	1.26×	1.30×	1.32×	(2.34×

make k calls: at $\sigma = 0$ the asynchronous arm’s bandwidth never left $\epsilon_0 = 5$ (59 calls per rank), its reported posterior sits at $W_1 = 2.1$ while its archive is at 0.02, and re-reported at the order statistic it is at 0.01 (Table 4, lower block). Figure 5 therefore scores archives, and on a history of a few thousand evaluations the reported estimator is only as good as the bandwidth the schedule managed to reach: the transient of §7.4, met here on a cheap simulator.

7.2 Throughput converts into a tighter tolerance (C2)

Throughput is instrumental; what a practitioner buys is the tolerance the run reaches. Table 6 reads that off every benchmark with the one quantity that compares samplers on a common footing, the tolerance $\epsilon_{(100)}(n)$ at which exactly 100 of a method’s own draws would be accepted after n simulations (§6.4), and Fig. 4 shows the curves it is read from. Read at the same n — the synchronous arm’s own count — the asynchronous sampler is more efficient per simulation on three of the four benchmarks (1.06× on g-and-k, 1.30× on Lotka–Volterra, 1.6× on Cellular Potts) and 0.78× on the one-dimensional analytic target, the only benchmark on which it is worse per draw; the ratio rises with the cost of a simulation. The throughput ratio rises with it too, so on the two benchmarks where both exceed one they compound into the net effect at equal wall clock: 1.3× on Lotka–Volterra and 4.1× on Cellular Potts (not their literal product, since the curves are not power laws over the range and each ratio is a median over replicates). The curves add what a single budget cannot: on the two cheap benchmarks the asynchronous and synchronous curves have the same slope, so the per-simulation ratio is a constant factor, whereas on Cellular Potts both adaptive samplers steepen well beyond the n^{-1} that rejection ABC follows in this noise-dominated regime, so their advantage over it widens with every doubling of the budget.

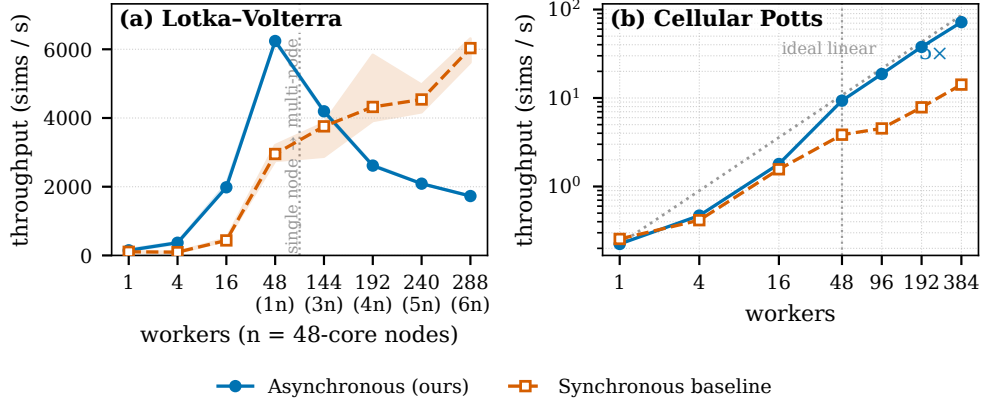


Fig. 3 Strong scaling at a fixed wall-clock budget, asynchronous against a synchronous baseline given a population of $\max(100, W)$ so that it can occupy every core. **Left:** Lotka–Volterra (4 ms per simulation; 180 s budget; medians and inter-quartile ranges over five replicates). With almost no simulation to distribute, the asynchronous method’s per-arrival coordination grows with the worker count: throughput peaks on one node and declines across multi-node jobs, and the baseline overtakes it beyond three nodes (§7.3). **Right:** Cellular Potts 50^3 in the benchmark’s earlier single-simulation configuration (5.1 s per evaluation, Appendix E; 1800 s budget; log–log). Real per-evaluation cost amortises the coordination and the asynchronous method tracks ideal linear scaling (dotted) to eight nodes at $\approx 97\%$ parallel efficiency from one node to eight (84% from a single worker). The gap against pyABC at this size is mostly pyABC’s per-generation overhead, not the barrier: the barrier’s own share at 50^3 is $1.2\times$ (Table 5), and at the 80^3 runtime spread it would be a predicted $2.1\times$ at eight nodes (§7.1).

Table 6 Tolerance at a matched budget on every benchmark ($k = 100$ order statistic of each method’s own discrepancies; five replicates; medians; completed simulations only). The three cheap benchmarks ran 48 ranks as twelve per node on four nodes, Cellular Potts on one node (Appendix D). Per-simulation: ϵ of the synchronous arm over the asynchronous arm at the synchronous arm’s own simulation count. Equal wall clock: the same ratio at the end of both runs. Both rise with the cost of a simulation. Replicate ranges, pairing replicates by index (benchmarks in table order): per-simulation 0.77–1.12, 1.02–1.15, 1.08–1.51, 1.14–1.94; equal wall clock 0.21–0.28, 0.85–0.92, 1.02–1.43, 3.1–6.6.

Benchmark	cost/sim	sims (async / sync)	throughput	per-simulation	equal wall clock
Gaussian mean	0.1 ms	1.31M / 4.61M	$0.28\times$	$0.78\times$	$0.24\times$
g-and-k	2 ms	1.75M / 3.81M	$0.46\times$	$1.06\times$	$0.88\times$
Lotka–Volterra	4 ms	1.51M / 1.56M	$0.96\times$	$1.30\times$	$1.30\times$
Cellular Potts	13 s	12,900 / 5,400	$2.39\times$	$1.6\times$	$4.09\times$

Cellular Potts, both arms fixed.

On the real simulator the decomposition is measured on the fixed production run (Table 8): the asynchronous arm completes $2.4\times$ the simulations of the baseline in the same hour (99.8% worker utilisation against 42.8%; by the factorisation of §7.1 about $1.2\times$ of this is the barrier and the rest pyABC’s per-generation overhead against a 13 s evaluation, which is why $2.0\times$ is the figure we attribute to the barrier), reaches a $1.6\times$

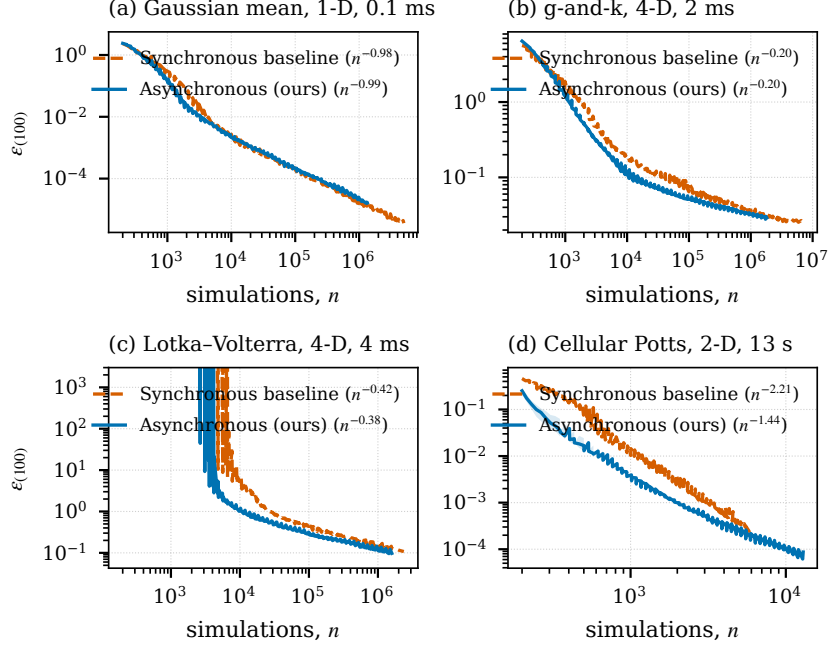


Fig. 4 Tolerance against simulations spent, $\epsilon_{(100)}(n)$, asynchronous against the matched synchronous baseline on every benchmark (medians and replicate range over five replicates; the slope over the last decade of n in the legend). The vertical gap at equal n is the per-simulation efficiency, the horizontal extent is what the wall clock bought. On Lotka–Volterra 98% of simulations return the extinction fallback, so $\epsilon_{(100)}$ sits on that plateau until about 5000 draws have produced 100 survivors; the plateau is cut off.

tighter tolerance at the baseline’s own simulation count, and ends $4.1\times$ tighter at equal wall clock (7.6×10^{-5} against 3.1×10^{-4}). The per-simulation ratio is read at 5,400 simulations and is not budget-free either: over its last decade the baseline’s curve is the steeper ($n^{-2.2}$ against $n^{-1.4}$, Fig. 4d), so the gap would close near 10^4 simulations if both slopes held. A fairly resourced rejection sampler — 13,000 prior draws on all 48 ranks, the same number of simulations as one asynchronous replicate — reaches 2.9×10^{-3} , $39\times$ looser. The control run with the shipped starting bandwidth gives $7.1\times$ at equal wall clock rather than $4.1\times$, and the difference is a warning about matched comparisons: the starting bandwidth is shared by construction, and fixing it improved the baseline’s tolerance more than ours ($4.5 \rightarrow 3.1 \times 10^{-4}$ against $6.3 \rightarrow 7.6 \times 10^{-5}$), so the larger figure rests on a handicapped baseline and we do not use it. At 80^3 the same run gives $2.1\times$ the simulations and $2.1\times$ per simulation, and a $17\times$ tighter tolerance at equal wall clock; we quote the first two and not the third, because at a one-hour budget both arms sit on the steep early part of their curves (doubling the asynchronous arm’s 481 simulations to 988 tightened its tolerance $7.7\times$), and a ratio that depends on the budget that sharply is not a property of the method.

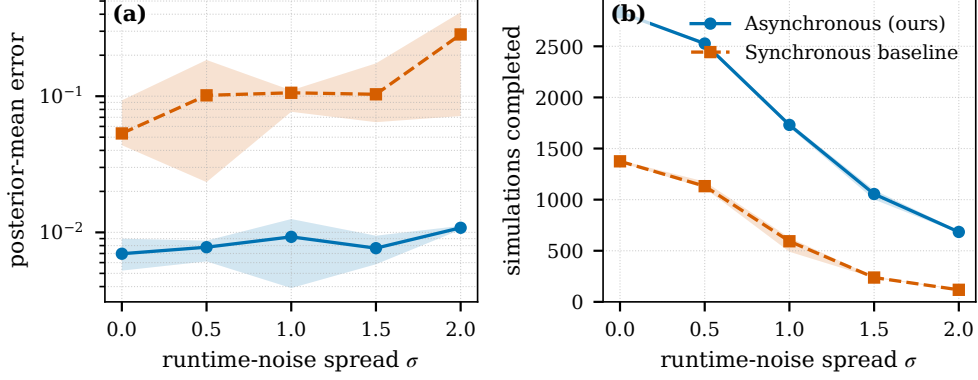


Fig. 5 Inference per unit wall clock under runtime heterogeneity (Gaussian mean, analytic posterior, 48 workers, 60 s budget, five replicates; bands are inter-quartile ranges). Left: error of the archive mean (the unweighted top- k archive, not the reported estimator; §7.1) to the analytic posterior mean, against the runtime spread σ . Right: simulations completed within the budget. The barrier starves the synchronous baseline as σ grows; the asynchronous method’s error stays flat.

The synthetic instance.

The heterogeneity study shows the same conversion where the runtime law is known (Fig. 5): at a fixed 60 s budget on the Gaussian benchmark the synchronous baseline completes fewer simulations as σ grows and its archive’s posterior-mean error to the analytic posterior climbs from ≈ 0.05 to ≈ 0.28 , while the asynchronous arm completes several-fold more and its archive holds ≈ 0.007 – 0.011 . These are archive means; the *reported* estimators of both arms on this campaign are a different matter, taken up in §7.1.

7.3 Where the advantage turns on (C3)

The asynchronous method pays a roughly fixed cost per arrival — rebuild the $\mathcal{O}(k)$ proposal, weight the arrival against $S+1$ proposal snapshots, update the archive, and exchange the arrival with every other worker — and that cost is what decides where it pays. Figure 6 plots the throughput and per-simulation ratios of Table 6 against the cost of one simulation. Both are monotone in that cost, but only one has a mechanism: the throughput ratio pits a fixed per-arrival cost against a growing per-simulation one, whereas the per-simulation ratio is a statistical property of four inference problems that differ in dimension, summaries and noise, and its ordering is as consistent with problem dependence as with cost, so the rule below rests on the throughput ratio alone. The throughput ratio is $0.28\times$ at 0.1 ms, $0.46\times$ at 2 ms, $0.96\times$ at 4 ms and $2.4\times$ at 13 s, so parity is reached at about 4 ms per simulation and the net advantage at equal wall clock turns positive between 2 and 4 ms. That boundary belongs to the placement and budget it was measured under. The production campaign of the cheap benchmarks ran its 48 ranks twelve per node on four nodes (Appendix D), where every arrival crosses the network; on one packed node the same Lotka–Volterra configuration completes 6240 simulations per second on the asynchronous arm against 2952 on the

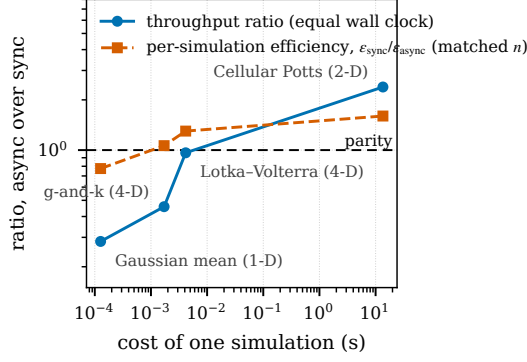


Fig. 6 Throughput ratio and per-simulation efficiency of the asynchronous method over the matched synchronous baseline, against the cost of one simulation, one point per benchmark (Table 6; 48 ranks placed twelve per node on four nodes). Both rise with the cost; throughput parity is reached at about 4 ms per simulation at these budgets. Only the throughput ratio has a cost mechanism (§7.3).

baseline over 180 s (Table F3), a ratio of 2.1 where the four-node production run gives 0.96, and within the production run the asynchronous throughput rises from 840 to 3550 simulations per second between its first and last minute. The crossover moves down with tighter placement, and Fig. 6 fixes it for one. Below that the two cheap benchmarks are the measured wrong side of the line: on the Gaussian mean the asynchronous method finishes with a tolerance $4\times$ looser than the baseline’s at equal wall clock, not because it samples worse — per simulation it is within $0.78\times$ — but because the baseline completes $3.5\times$ more simulations of a 0.1 ms simulator. Where the simulations are that cheap a barrier costs nothing worth removing.

The same cost sets the scaling boundary on a cheap simulator. On Lotka–Volterra (4 ms) an instrumented run puts a worker inside the simulator 25% of its time at 48 workers, 8% at three nodes and 2% at six (Appendix F): the exchange of every arrival with every worker grows with the worker count, so aggregate throughput peaks on one node (6240 simulations per second at 48 workers) and declines across fully packed multi-node jobs, and a synchronous baseline given a population equal to the worker count overtakes it beyond three nodes (Fig. 3, left). Enlarging the archive tenfold moves that crossover in from beyond three nodes to three, because the per-arrival proposal reconstruction is $\mathcal{O}(k)$ (Appendix F). On Cellular Potts the same coordination is invisible: at 13 s per evaluation the asynchronous method tracks linear scaling to eight nodes (Fig. 3, right). The rule this gives a practitioner is simple. Above a few milliseconds per simulation the barrier is the larger cost and its removal pays by the barrier’s cost of §7.1; below it, or on a cheap simulator spread over several nodes, the coordination is, and a generation-staged sampler with a population sized to the machine is the better tool.

Table 7 Simulation-based calibration of the reported posterior (1000 trials; empirical coverage of central intervals at the nominal levels; Monte Carlo standard error ± 0.007 – 0.016). Nominal is best.

Target	Method	0.50	0.80	0.90	0.95
Gaussian mean (1-D)	asynchronous	0.51	0.80	0.90	0.94
	synchronous baseline	0.35	0.63	0.75	0.83
Bimodal $\mathcal{N}(\theta^2, \sigma^2)$ (1-D)	asynchronous	0.50	0.82	0.91	0.96
	synchronous baseline	0.45	0.82	0.90	0.95
g-and-k (4-D), A	asynchronous, full history	0.62	0.91	0.96	0.99
	synchronous baseline	0.54	0.82	0.91	0.95
g-and-k (4-D), B	asynchronous, full history	0.54	0.83	0.91	0.96
	synchronous baseline	0.45	0.74	0.85	0.90
g-and-k (4-D), g	asynchronous, full history	0.58	0.88	0.95	0.98
	synchronous baseline	0.51	0.82	0.91	0.95
g-and-k (4-D), k	asynchronous, full history	0.56	0.83	0.92	0.96
	synchronous baseline	0.47	0.74	0.86	0.92
g-and-k (4-D), A	asynchronous, top- k archive	—	—	0.85	0.89

7.4 The reported posterior: calibration, recovery, and what limits it (C4)

Calibration.

Table 7 gives simulation-based calibration of the reported full-history estimator (1000 trials, matched simulation budget). On the one-dimensional Gaussian mean it tracks the nominal level at every level (0.51/0.80/0.90/0.94); the synchronous baseline under-covers at every level (0.35/0.63/0.75/0.83), and with a Monte Carlo standard error of ± 0.007 – 0.016 per entry that ordering is resolvable, not noise. On a bimodal target ($y_i \sim \mathcal{N}(\theta^2, \sigma^2)$, the case a top- k archive is most at risk of collapsing onto one mode) the archive retains both modes in 86% of trials and coverage is 0.50/0.82/0.91/0.96 (the baseline: 0.45/0.82/0.90/0.95). On the four-dimensional g-and-k model the estimator is *conservative* on two parameters and near nominal on the other two: 0.62/0.91/0.96/0.99 for A and 0.58/0.88/0.95/0.98 for g , against 0.54/0.83/0.91/0.96 for B and 0.56/0.83/0.92/0.96 for k ; the rank histograms of A , g and k are not uniform at the 1% level, B ’s is. The synchronous baseline is near nominal on A and g and under-covers B and k , with no uniform rank histogram. That is a reporting-support effect, not a weighting failure: the top- k archive alone would under-cover (0.85/0.89 at the 0.90/0.95 levels), the full history over-covers, coverage rises monotonically through nominal as the reported support grows from one to the other, and doubling the simulation budget leaves it unchanged. A sweep of the archive size and snapshot buffer on a d -dimensional Gaussian target (Appendix F) finds the buffer’s effect saturating by $S = 5$ in one and two dimensions and non-monotone above it at $d = 8$, and the archive size mattering, non-monotonically, with a genuinely mis-calibrated region at $d = 8$, $k = 50$; $k = 100$ is within 7% of the fastest archive and the best calibrated of those swept.

Table 8 Cellular Potts 50^3 , five replicates per adaptive method and one rejection run, 3600 s on 48 workers (means $\pm$ s.d. for contraction, medians otherwise). The first row is the asynchronous method with the shipped starting bandwidth (10, twenty times the prior-predictive median discrepancy), the rest the production configuration (0.1). Rejection ABC is one run of 13,000 prior draws on all 48 ranks, keeping the best 100. Contraction is $1 - \text{sd}/\text{sd}_{\text{prior}}$; the truth lies inside the central 90% interval of every run.

	sims	util.	$\epsilon_{(100)}, \text{end}$	contraction		ESS
				division rate	cell volume	
Asynchronous, $\epsilon_0 = 10$	13,300	99.8%	6.3×10^{-5}	$84 \pm 6\%$	$13 \pm 14\%$	645
Asynchronous, $\epsilon_0 = 0.1$	12,900	99.8%	7.6×10^{-5}	$92 \pm 1\%$	$81 \pm 1\%$	303
Synchronous baseline, $\epsilon_0 = 0.1$	5,400	42.8%	3.1×10^{-4}	$91 \pm 1\%$	$72 \pm 4\%$	66
Rejection ABC, best 100 of 13,000	13,000	—	2.9×10^{-3}	91%	67%	100

Recovery against a reference posterior.

Where a reference exists, Fig. 7 scores each method on the posterior it reports, replayed over wall-clock prefixes. On the Gaussian mean the asynchronous estimator sits at $W_1 = 0.008\text{--}0.016$ from the analytic posterior at every checkpoint, the baseline’s final population at $0.019\text{--}0.028$ (uniformly weighted; its history was not kept), and a fairly budgeted rejection sampler’s best 100 of 20,000 draws at 0.011 — on a one-dimensional target with that many draws, rejection is as good as the reported estimator, and the advantage of an adaptive proposal is a story of dimension and cost, not of the analytic case. On the four-dimensional g-and-k the order reverses: the baseline ends at 0.029, the asynchronous estimator at 0.069, rejection at 0.47. The lower panels say why. The reported posterior’s effective sample size is 238–296 at every checkpoint of the Gaussian run while the history grows from 10^5 to 1.3×10^6 particles, and a controlled sweep puts ESS/k between 1.9 and 3.6 over $k \in \{50, 100, 200\}$ and n up to 8×10^4 : reporting at the tightest bandwidth reached, while that bandwidth is set by ESS-retention bisection against the archive, leaves an effective support of a few multiples of k however many particles are evaluated. In one dimension that support is small and stable; in four it is small and erratic — 7 to 2987 across checkpoints and replicates — and accuracy follows it (log–log correlation -0.80 with W_1). The four-dimensional deficit is intermittent weight degeneracy, not bias, and the ceiling binds exactly when the target is identified enough for the bandwidth to tighten.

A known truth on the real simulator.

On the two-parameter Cellular Potts setup — one synthetic dataset with a known truth — all three methods recover it (Fig. 8, Table 8). The division rate is well identified by every method (91–92% contraction against the prior); the cell volume separates them: $81\% \pm 1\%$ for the asynchronous estimator, $72\% \pm 4\%$ for the baseline’s last generation with pyABC’s weights, 67% for the best 100 of 13,000 rejection draws, with the truth inside the central 90% interval of every run (five per adaptive method, one rejection run); on a single dataset that is containment, not coverage. The asynchronous estimator is the tightest and carries the largest effective sample (303 against the baseline’s 66 of 100, medians).

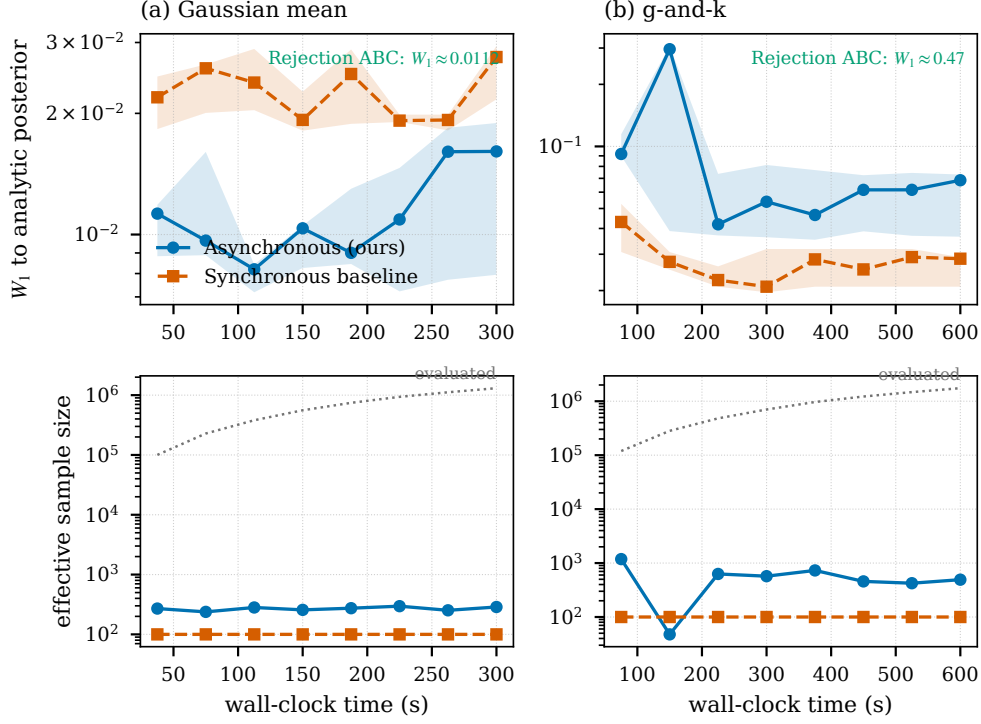


Fig. 7 What the reported estimator recovers, and with how many effective particles. Each method is scored on the posterior it reports — the asynchronous method’s retroactive AMIS estimator replayed over the prefix completed by each checkpoint, the synchronous baseline’s last completed generation (uniformly weighted; no history was kept), rejection ABC’s 100 best draws of its budget — against the analytic posterior (a) and the g-and-k reference posterior (b). Top: W_1 , median and inter-quartile range over five replicates. Bottom: the effective sample size of the reported posterior, with the evaluated-particle count dotted; the history grows by more than an order of magnitude and the effective sample does not follow it.

The starting bandwidth.

The first row of Table 8 is the same sampler with the starting bandwidth the benchmark shipped with, $\epsilon_0 = 10$, and it reports $13\% \pm 14\%$ on the cell volume — erratic as well as poor. The sampler’s draws are the same in both rows (the end tolerance is 6.3×10^{-5} against 7.6×10^{-5}); what differs is the bandwidth the estimator reports at. The schedule walks down from ϵ_0 at a bounded rate — each rank searches once per k of its own calls and halves at most — so on an expensive simulator that transient is the whole run: 3000 evaluations on 48 ranks are 62 calls per rank, and at $\epsilon_0 = 10$ the bandwidth had not moved after them and ended $321\times$ above the tolerance the run’s own draws supported; at 0.1 it ends $2\times$ above it. On a simulator that affords millions of evaluations the transient is invisible, which is why the cheap production runs never showed it; it binds in exactly the regime the method is for, and on any run in which no

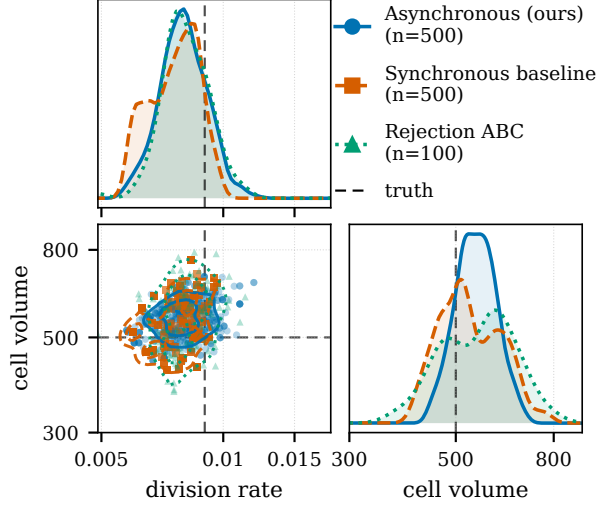


Fig. 8 Cellular Potts posterior (division rate, cell volume) on the fixed production run, replicate 0: the asynchronous AMIS estimator (500 resampled draws), the synchronous baseline’s last completed generation with pyABC’s weights (500 resampled draws), and the 100 best of 13,000 rejection draws on 48 ranks. Axes are the log-uniform prior coordinates ticked in physical units; dashed lines mark the truth. Every method recovers the division rate; the asynchronous estimator is the tightest on the cell volume.

rank makes many multiples of k calls whatever the simulator costs: on the 60 s heterogeneity campaign of §7.1 the bandwidth never left ϵ_0 , and under a barrier it stops at its throttle points because the history rebuilds that otherwise trigger a search do not occur. The re-report at the order statistic recovers those histories too (Tables 3 and 4); a search triggered by the growth of the shared history rather than by each rank’s own call count would remove the failure at its source, and is the implementation change we recommend. The rule that transfers is to set ϵ_0 from a quantity measurable before the run and without the truth, the prior-predictive median discrepancy (0.46 here, from 2000 prior draws): about a fifth of it. The retroactive alternative — re-reporting a stored history at the k -th order statistic of its own losses, at no simulation cost — reaches the same posterior (93%/85% on replicate 0) and is the check we apply, but an a priori setting is the less contestable fix.

The archive size.

The effective sample size is a few multiples of k , so raising k looks like the obvious lever; it is not a free one. Re-reporting one history at the order statistic for $k = 10$ to 1000 moves the cell-volume contraction monotonically from 85% to 78% with coverage held, because the reported bandwidth loosens with k . Running the sampler with $k = 300$ is worse than that arithmetic: the bandwidth search interval defaults to k , so the schedule never left ϵ_0 in an hour and the cell volume came out at 7%; with $k = 30$ contraction matches $k = 100$ but one of two replicates misses the truth on the division rate. On the cheap benchmarks the picture differs — the bandwidth loosens only as $k^{0.28}$ on the four-dimensional g-and-k, and $k = 100$ is Pareto-optimal for throughput

and calibration on Lotka–Volterra (Appendix F) — so the archive size is a benchmark-dependent choice, to be made by checking calibration and the bandwidth the schedule reaches, not by a rule. The two settings together bound what the estimator delivers: the starting bandwidth decides whether the reported posterior reaches what the draws support within the budget, and the archive size decides how many effective particles it reports there. Where a production run cannot afford a calibration run, both are readable from its own history after the fact.

The 80³ configuration is the reminder that both bind harder as simulations get more expensive: at a one-hour budget of about 1000 evaluations every method reports a weak posterior there (50%/15% asynchronous, 52%/11% baseline, 78%/53% for the best 100 of 1000 prior draws), and we use that size for the systems measurement of §7.1 only.

8 Discussion

The results give a practitioner four rules, each backed by a measurement rather than an argument.

Use it above a few milliseconds per simulation. Below that the per-arrival coordination of a single shared archive costs more than a generation barrier does, and a generation-staged sampler with a population sized to the machine is the better tool (§7.3). Above it the barrier is the larger cost, and its removal pays by the barrier’s cost, which for identically distributed runtimes is the straggler factor of the workload and grows with the worker count and with the spread of simulation times — and a simulator’s spread tends to grow with its size, as the Cellular Potts model’s did from 0.05 to above 0.2.

Predict the gain before paying for it. A production project with an unknown posterior cannot afford a synchronous reference run to find out what asynchrony would have bought. It does not need one: the barrier’s cost is $W/\mathbb{E}[\max_{i \leq W} D_i]$ against the asynchronous throughput, computable from the run’s own per-evaluation timing (§7.1). Two cautions come with it. The timing sample must be longer than the tail of the runtime distribution, or the prediction undershoots; and a persistently slow worker contributes its own evaluation time, not its wall-clock span.

Set the starting bandwidth from the prior-predictive discrepancy scale. On an expensive simulator the whole run is spent inside the bandwidth transient, and a starting value twenty times the prior-predictive median costs the reported posterior most of its contraction on the weakly identified parameter while changing nothing about the draws (§7.4). About a fifth of the median discrepancy of a few thousand prior draws is the rule that transferred; re-reporting a stored history at the order statistic of its own losses is the check, and so is confirming that the schedule ran at all: a rank that made fewer than k calls never searched.

Keep the archive small and check calibration. The reported posterior’s effective sample size is a few multiples of the archive size, and enlarging the archive loosens the reported bandwidth and lengthens the transient; $k = 100$ was never far from the best choice on any benchmark, and the calibration checks of §7.4 are cheap to repeat.

The effective-sample-size ceiling is where the method’s headroom is. Barrier removal delivers simulations reliably and predictably; converting all of them into posterior accuracy is limited by a reporting rule — report at the tightest bandwidth reached, chosen by ESS retention against the archive — that we adopted for its simplicity, and the next work on this method belongs there rather than in the scheduler.

9 Limitations

- *Theory.* Consistency holds up to a fidelity ratio r between the density the sample was drawn from and the denominator the estimator divides by; $r \equiv 1$ recovers the smooth-ABC posterior, two of its four factors are measured at about one percent in total variation, and the other two are bounded, not measured (Appendix A). The central limit theorem covers damped-adaptation variants; the configuration we run measurably misses its rate condition, for a reason that lies in the inference problem (a weakly identified direction) rather than in asynchrony. Under asynchronous execution the existence of an adapted filtration is assumed, and the reported runs leave the bandwidth floor unset, so $\epsilon_\infty > 0$ is assumed for them.
- *Selection by completion time.* Keeping every evaluated particle over-represents parameter regions that simulate quickly. A study that couples runtime to the parameter finds no measurable shift of the reported posterior mean at the couplings tested, with or without AMIS reweighting, and a drain-after-deadline control finds the deadline censoring negligible there (Appendix F); we do not correct for it and do not claim the reweighting cancels it.
- *What the twin does not settle.* On Cellular Potts the twin’s simulations ran $1.5\text{--}2.3\times$ longer than the asynchronous arm’s for reasons a barrier cannot cause; we factor that out rather than explain it. The $2.1\times$ predicted for 384 workers at the 80^3 runtime spread is not measured: scaling to eight nodes was run only at 50^3 , where pyABC’s per-generation overhead dominates the gap. The twin deadlocks intermittently at ≥ 192 ranks and needed retries. It synchronises timing only and does not truncate histories; the collapse of its reported posterior at the schedule’s bandwidth is the scheduler’s per-rank throttle, not the estimator, and re-reporting undoes it (§7.1).
- *Two Cellular Potts configurations.* The 50^3 twin and scaling campaigns, and the theory diagnostics of Appendix A, ran the benchmark’s earlier configuration (division rate and motility, single simulations of 5.1 s); the posterior results and the 80^3 systems comparison ran the two-parameter production configuration (Appendix E). The systems conclusions transfer through the runtime distribution, which is all the predictor consumes; the theory diagnostics have not been repeated on the production history.
- *Placement.* The production campaign of the three cheap benchmarks placed its 48 ranks twelve per node on four nodes; on one packed node the same Lotka–Volterra configuration runs $2.5\times$ faster on the asynchronous arm, so the cost boundary of §7.3 is a property of that placement and budget, not of the simulator alone.
- *Baseline configuration.* The production comparisons give pyABC a population of 100 on 47 simulating ranks, which idles by construction; a population that is a

multiple of the worker count, and pyABC’s look-ahead mode, were not run on the production benchmarks.

- *Baseline weights.* On the Gaussian and g-and-k campaigns the synchronous baseline’s populations are scored uniformly weighted because their pyABC histories were not kept; on Cellular Potts, where they were, the weights move contraction by three to four points and coverage not at all.
- *The benchmark’s scale.* The Cellular Potts runs cost seconds to minutes per evaluation against hours in production, infer two parameters — the two directions its summaries resolve — and use synthetic data with a known truth. They establish the method and the predictor; they do not measure a production posterior. Lotka–Volterra sends 98% of simulations to extinction and is not scored for posterior quality.
- *The two settings.* The starting bandwidth and the archive size govern what the reported posterior delivers on an expensive simulator, and the archive-size sweep varied the bandwidth search interval with it; the two effects are separable now that the interval is exposed, and were not separated here. The snapshot buffer is fixed at $S = 20$.

10 Conclusion

A generation barrier costs the ratio of the barrier-free throughput to what a generation allows — the straggler factor of the workload when runtimes are identically distributed — and that is a quantity, not a slogan: predicted from the asynchronous run’s timing alone, it matched a barrierized twin of our own sampler within one per cent under a persistent straggler, within twenty per cent under lognormal heterogeneity, and within ten per cent on a real tissue simulator, across two and a half orders of magnitude. The generation-free sampler we propose removes that cost, reports an estimator that is consistent up to a measured fidelity ratio, calibrated in one dimension and conservative in four, and replayable from its log, and converts the simulations it gains into a tighter tolerance wherever a simulation costs more than a few milliseconds — $2.1\text{--}2.4\times$ the simulations, of which the barrier accounts for $2.0\times$, $1.6\times$ per simulation and $4.1\times$ at equal wall clock on the tissue simulator, with the known truth recovered. What it does not do is turn every simulation into posterior accuracy: the reported posterior’s effective sample is a few multiples of the archive size, and the starting bandwidth must be set from the prior-predictive scale on an expensive simulator. Both are properties of the reporting rule, and that is where the next step lies. For simulator-based inference on machines where the simulations are expensive and unequal, we think generation-free execution is the right model, and we have tried to give the practitioner the two things needed to adopt it: a prediction of what it will buy, from their own timing, and the settings on which what it buys depends.

Acknowledgements. *To be added in the final version.*

Declarations

Funding

To be added in the final version.

Conflict of interest/Competing interests

The authors declare no competing interests.

Ethics approval and consent to participate

Not applicable.

Consent for publication

Not applicable.

Data availability

The repository accompanying this paper already contains, for every figure and table, the per-figure summary data it is drawn from (37 CSV files under `experiments/data/paper_figures/`), the generator that produces each one from the campaign output (`experiments/scripts/make_*.py`, 27 of them, most with a `--refresh` path that re-derives its CSV and all with a default path that draws from the committed copy without touching the cluster; the predictor rows and the 80^3 systems point are vendored from the study scripts), and the committed output of every diagnostic that produces a number quoted in the text (`experiments/data/diagnostics/`). Every figure and table can therefore be regenerated, and every quoted number checked, from the repository alone. What the repository does *not* contain is the raw per-particle records themselves — they reach ≈ 30 GB for the Gaussian-mean campaign alone — together with the per-rank phase-timing logs and the resolved run configurations; those will be deposited in a public archive upon publication, and the same generators reproduce the committed summaries from them.

Materials availability

Not applicable.

Code availability

The full implementation — the Propulate propagator, the matched-kernel pyABC baseline, the analysis and plotting scripts, and the test suite that pins the properties the text relies on (that the plot-thinning cannot reach a reported number, that a stored run replays to its own reported posterior bit-identically, and that the point-mass distance ranks a collapsed posterior above a correct one) — will be released in a public repository upon publication.

Author contribution

To be added in the final version.

Appendix A The central limit theorem and the fidelity ratio

This appendix carries the parts of the analysis that the main text states in one paragraph: what is new in the argument, the central limit theorem and its per-run confidence intervals, a bound on how far the tilt by r can move the target, and the decomposition and partial measurement of r on stored runs.

What is and is not new here.

The r -tilted limit of Theorem 1 is, in itself, the elementary observation that self-normalized importance sampling against a mismatched denominator converges to a correspondingly tilted target; stated for a fixed proposal it would not merit a theorem. Three things are not routine. The scheme is adaptive at every draw, so the argument runs through a martingale array rather than an i.i.d. one, and the available AMIS results do not cover it: [6]’s convergence argument is heuristic, and [14] restricts the adaptation so that the weighting stage decouples, which a single-arrival scheme does not do. The prior floor in the *denominator* bounds every weight by $1/\delta$ with no moment or density-ratio condition, which is what makes the conditional Lindeberg step trivial and lets a local archive proposal — for which the usual two-sided ratio bound simply fails — be used at all. And the departure from the ideal is carried as r inside the statements, then decomposed and partly measured, rather than assumed away. We do not claim to weaken the hypotheses of any existing AMIS theorem; we claim a different bargain, in which the implementation’s infidelity is a quantity in the conclusion instead of a condition on the algorithm.

Theorem 3 (central limit theorem). *Let Assumptions 1–4 and 5(i)–(ii) hold. Then for every bounded measurable f ,*

$$\sqrt{n}(\hat{\pi}_n(f) - \pi_{\epsilon_\infty}^r(f)) \longrightarrow \sigma_f \mathcal{Z} \quad \text{stably}, \quad \mathcal{Z} \sim \mathcal{N}(0, 1) \text{ independent of } \mathcal{F}_\infty, \quad (\text{A1})$$

where the $\mathcal{F}_\infty$ -measurable σ_f^2 is

$$\sigma_f^2 = \frac{1}{Z_r^2} \int_{\Theta} \pi(\theta)^2 (f(\theta) - \pi_{\epsilon_\infty}^r(f))^2 L_{\epsilon_\infty}^{(2)}(\theta) \frac{\tilde{q}_\infty(\theta)}{\bar{q}_\infty(\theta)^2} d\theta, \quad Z_r = \int_{\Theta} \pi L_{\epsilon_\infty} r. \quad (\text{A2})$$

A caveat belongs with this statement rather than after it: the rate conditions (10) are measurably violated in the configuration our experiments use (§A.1), so Theorem 3 should be read as covering the damped- or frozen-adaptation variants, not those runs. At $r \equiv 1$ the variance collapses to $Z_{\epsilon_\infty}^{-2} \int \pi^2(f - \pi_{\epsilon_\infty} f)^2 L_{\epsilon_\infty}^{(2)} / \bar{q}_\infty$, the familiar self-normalized form. Its structure is readable in either case. Because $L_\epsilon^{(2)} \geq L_\epsilon^2$ by Jensen, (A2) always exceeds the variance of the corresponding *noise-free* importance sampler that could evaluate L_{ϵ_∞} exactly; the gap $L_\epsilon^{(2)} - L_\epsilon^2 = \text{Var}(K_\epsilon(\rho) \mid \theta)$ is precisely the price of running one simulation per proposed parameter. For the Gaussian kernel $K_\epsilon(\rho)^2 = K_{\epsilon/\sqrt{2}}(\rho)$, so $L_\epsilon^{(2)} = L_{\epsilon/\sqrt{2}}$ and that price is computable from the same likelihood at a tightened bandwidth. Convergence is stable rather than merely distributional because $\tilde{q}_\infty$, and hence σ_f^2 , may be random; it reduces to an ordinary $\mathcal{N}(0, \sigma_f^2)$ limit when the limiting proposal is deterministic. Stability is also what makes the theorem usable, because it survives division by a consistent variance estimate:

Corollary 4 (per-run confidence intervals). *Under the assumptions of Theorem 3, and provided $\sigma_f^2 > 0$, with $\hat{\sigma}_n^2 := n \sum_i W_{i,n}^2 (f(\theta_i) - \hat{\pi}_n(f))^2 / (\sum_i W_{i,n})^2$ we have $\hat{\sigma}_n^2 \rightarrow \sigma_f^2$ a.s. and $\sqrt{n}(\hat{\pi}_n(f) - \pi_{\epsilon_\infty}^r(f)) / \hat{\sigma}_n \Rightarrow \mathcal{N}(0, 1)$. The interval $\hat{\pi}_n(f) \pm z_{1-\alpha/2} \hat{\sigma}_n / \sqrt{n}$*

therefore has asymptotic level $1 - \alpha$ and is computable from the stored weights alone, at no extra simulation cost.

Proposition 5 (how far the tilt can move the target). *Let $A \subseteq \Theta$ and suppose $|r - 1| \leq \eta_A$ on A and $|r - 1| \leq 1$ off it. Then with $\zeta := \eta_A + \pi_{\epsilon_\infty}(A^c)$,*

$$\zeta < 1 \implies \|\pi_{\epsilon_\infty}^r - \pi_{\epsilon_\infty}\|_{\text{TV}} \leq \frac{\zeta}{1 - \zeta}. \quad (\text{A3})$$

More sharply, ζ may be replaced by $\zeta_0 = \pi_{\epsilon_\infty}(|r - 1|)$. Estimating ζ_0 from a stored run requires untilting: the normalized reported weights $\hat{w}_i$ target π^r , not π_{ϵ_∞} , so $\sum_i \hat{w}_i |r(\theta_i) - 1|$ estimates $\pi^r(|r - 1|)$; weights proportional to $W_{i,n}/r(\theta_i)$ target π_{ϵ_∞} and give $\hat{\zeta}_0$.

A.1 What r is for this implementation

Everything specific to the code enters through r . To attribute it, walk from the density actually sampled from down to the function actually divided by, changing one thing at a time. Let $\bar{q}^{(1)} = \frac{1}{n} \sum_{i \leq n} q_i$ use the *nominal* archive proposals — the truncated mixtures the propagator intends, with exact normalizing constants and without the rejection cap or the underflow redraws; let $\bar{q}^{(2)}$ replace those constants by the implementation’s product-of-marginals values; and let $\bar{q}^{(3)}$ be the m -point draw-proportional version of $\bar{q}^{(2)}$ carrying the observed bootstrap share ν_n . Assume each of the four displayed ratios has a uniform limit, finite and bounded away from zero — which fidelity alone does not give, since (9) constrains only their product. Writing $\bar{q}$ for the reported denominator (7),

$$r = \underbrace{\lim \frac{\bar{q}^*}{\bar{q}^{(1)}}}_{\text{(d) cap, redraws, asynchrony}} \cdot \underbrace{\lim \frac{\bar{q}^{(1)}}{\bar{q}^{(2)}}}_{\text{(c) normalizer}} \cdot \underbrace{\lim \frac{\bar{q}^{(2)}}{\bar{q}^{(3)}}}_{\text{(b) quadrature}} \cdot \underbrace{\lim \frac{\bar{q}^{(3)}}{\bar{q}}}_{\text{(a) floor}}.$$

The product telescopes for any positive intermediates, so the content is in the choice of intermediates, not the algebra: a different insertion order attributes the total differently, and the factors are a decomposition of one quantity rather than four independent errors. Note also what the first factor carries under asynchronous execution. If completion times steer scheduling, $\tilde{q}_i$ is the enlarged-field conditional law rather than the density the propagator emitted (Assumption 4), and that discrepancy sits in $\bar{q}^*/\bar{q}^{(1)}$ alongside the cap and the redraws. The four factors are then not a complete attribution of r to implementation details — one of them is partly a property of the schedule.

(a) *The prior floor.* When genuine bootstrap draws carry less than a δ share of the history, $w_n = \delta > \nu_n$ and the denominator holds more prior mass than the sample does. With $u = q_\infty/\pi$ the limiting proposal-to-prior ratio, this contributes $r_{\text{floor}} = (\nu_\infty + (1 - \nu_\infty)u)/(\delta + (1 - \delta)u)$, so $r_{\text{floor}} - 1 = (\delta - \nu_\infty)(u - 1)/(\delta + (1 - \delta)u)$: it is exactly 1 when $\nu_\infty \geq \delta$, below 1 wherever the proposal is thinner than the prior, and $r_{\text{floor}} \rightarrow 1$ as $\delta \rightarrow 0$, i.e. as m grows. This is the one contribution available in closed form.

(b) *The snapshot quadrature.* The m -point mixture stands in for the n -point one, with error at most V_n/m where $V_n = \sum_{i < n} \|q_{i+1} - q_i\|_\infty$ is the total variation of the *reconstructed* proposal path; this controls the third factor above, not the fidelity of the whole. Fixing m therefore trades cost against fidelity; see the remark below.

(c) *The truncation constant.* Each mixture component is normalized by the product of its per-axis in-box masses, which is exact for a diagonal perturbation covariance and increasingly accurate as a component’s mass sits away from the faces of the box relative to its own scale, but an approximation otherwise.

(d) *Capped rejection, underflow redraws, and (asynchronously) the filtration gap.* A parent’s perturbation is rejection-sampled into the box for at most 992 attempts, so component j is realized with probability $W_j[1 - (1 - Z_j)^{992}]$ and the residual is emitted as a prior draw; and a candidate whose proposal-time denominator falls below 10^{-12} is redrawn up to five times, the last one being emitted regardless. The cap changes the law relative to unlimited rejection whenever $Z_j < 1$, through the residual probability $(1 - Z_j)^{992}$; 10^{-3} is the scale at which that becomes appreciable rather than a threshold below which it is absent. Under asynchronous execution this factor additionally carries the gap between the emitted proposal density and the conditional law of Assumption 4, which is a property of the schedule; unlike the other three contributions it is neither measured nor bounded structurally, and enters only through the assumption that this factor has a finite, positive limit.

Measured, in part.

What follows measures two of the four factors, not r . Two of the four — the prior floor and the snapshot quadrature — can be estimated rather than bounded, because $\hat{\pi}_n$ is a post-hoc pass: recomputing (7) at a much larger m and at the observed prior share gives a reference against which those factors are evaluated at every particle. The remaining two are *not* captured by such a reference: the truncation constant is bounded rather than measured, and factor (d) is bounded only for its cap-and-redraw part, its asynchronous component resting on the assumed factor limit alone. The figures below are therefore a partial accounting, not a lower bound on the total: the factors are multiplicative and an unmeasured one could in principle offset a measured one. On a Cellular Potts history from the benchmark’s earlier configuration (33,748 asynchronous evaluations across 48 workers, division rate and motility, two correlated dimensions, so that (c) is live; Appendix E), contribution (b) alone gives $\hat{\zeta} = 0.022$ with $r \in [0.92, 1.80]$, and the reported posterior sits 1.1% in total variation from the large- m reference, with posterior means within 0.2% and 1.1% of a standard deviation and the effective sample size essentially unchanged (0.213 against 0.228). On a one-dimensional Gaussian-mean history where the exact n -point mixture is computable, (a) and (b) together give $\hat{\zeta} = 0.042$ and a total-variation distance of 0.26% from the reference that removes (a) and (b), with the reported posterior 0.5% narrower — the tail-suppressing direction predicted by (a). Of contribution (d), the two events a stored history records — a rejection fallback and an exhausted redraw — did not occur once in 12,000 draws; a redraw that succeeded on a later attempt leaves no trace. We do not rule those out: we observe only that triggering one requires a candidate to land where the mixture it was drawn from has density below 10^{-12} , which is a necessary

condition for the event, not a bound on its probability. The bound (A3) is honest but loose against these direct measurements, by about a factor of two on the Cellular Potts run (2.3% against 1.1%) and by an order of magnitude on the one-dimensional history (4.4% against 0.26%), as sup-norm-free bounds valid for every f generally are.

Do the reported runs meet the stabilization conditions?

Assumption 5(ii) is the one place where we can do better than assume, because the proposal sequence is reconstructible from a stored history. Rebuilding q_τ along that same earlier-configuration Cellular Potts run and comparing against the terminal proposal separates two rates that behave very differently. The *per-step drift* $\|q_{\tau+1} - q_\tau\|_\infty$ decays as $\tau^{-0.92}$, and the archive’s membership changes only 832 times in 33,748 draws, tracking $k \log n$ at a ratio between 0.64 and 0.80 across the whole run — the rate a record-statistics heuristic predicts, since the k -th best of n draws improves with probability $\approx k/n$. The proposal path’s total variation is therefore $O(\log n)$, and the snapshot clause is comfortably met. The *distance to the limit* is another matter. Fitting $\|q_\tau - q_\infty\|_\infty \sim \tau^{-b}$ gives $b \approx 0.33$ over the full range and $b \approx 0.14$ – 0.23 when the fit is restricted away from the end, where using the terminal proposal as a stand-in for q_∞ biases b upward. Assumption 5(ii) needs $b > \frac{1}{2}$; at the measured values the aggregate $n^{-1/2} \sum_{i \leq n} \|q_i - q_\infty\|_\infty$ grows like $n^{0.17}$ to $n^{0.36}$ instead of vanishing.

We therefore do not claim Theorem 3 for the configuration these experiments run: its rate condition is not merely unverified there but measurably violated. Theorem 1 is unaffected, needing no rate. Two things temper the negative result and point somewhere specific. The measurement is one replicate of one benchmark over about a decade and a half of n , against a stand-in limit, so it is a diagnostic and not an asymptotic statement. The natural explanation — that the run leaves `min.tol` unset, so ϵ_n was still tightening and the archive was chasing a moving target — we tested and can reject. Running a two-dimensional Gaussian-mean benchmark serially under the same propagator, once with the data-driven schedule and once with the bandwidth pinned at the tolerance the first run reached a third of the way in (a floor the schedule then sat on for 68% of archive-phase draws), moves b by less than 0.07 and in opposite directions depending on the fit range: $0.61 \rightarrow 0.67$ restricted, $0.76 \rightarrow 0.69$ over the full range. Both arms comfortably satisfy $b > \frac{1}{2}$. So a tightening bandwidth does not prevent stabilization, and cannot be what breaks the Cellular Potts run.

What does explain it is the target. That earlier Cellular Potts configuration was assembled to exercise a realistic heterogeneous-runtime workload: its summaries, discrepancy and parameter ranges were not selected for identifiability, and its two parameters, division rate and motility, confound (Appendix E). The two-parameter production configuration was built to remove exactly that, and the measurement has not been repeated on it. Re-running the serial benchmark with that property deliberately induced — observing only $\theta_1 + \theta_2$, so the target is a ridge rather than a point — collapses the exponent from $b = 0.61$ – 0.76 to $b = 0.00$ – 0.29 , squarely onto the Cellular Potts range. A confounded target reproduces the failure in a *serial* run, so asynchronous replay is not needed to account for it and the adaptation rule is not what breaks.

The mechanism is transparent once stated: on a ridge the top- k archive slides along the flat direction indefinitely, because there is always a marginally better point further along it, so the proposal never settles and $\|q_\tau - q_\infty\|_\infty$ does not decay at the required rate. Assumption 5(ii) is thus a condition on the *inference problem* as much as on the sampler: it holds comfortably for a well-identified target under the same top- k rule and the same bandwidth schedule, and fails when the summaries do not identify the parameters. We therefore state Theorem 3 for the damped- or frozen-adaptation variants when the target is weakly identified, and note that for the identified benchmarks the measured rate satisfies the condition with room to spare.

Fixed versus growing m .

Contribution (b) is where the estimator’s cost meets its fidelity. With m fixed, the m -point mixture tracks the n -point one only if the proposal path settles, the error being V_n/m . Letting $m = m_n \rightarrow \infty$ with $V_n/m_n \rightarrow 0$ removes *this* contribution to r — it does not by itself deliver $r \equiv 1$, since the other three remain — and the proofs go through for any $m_n = o(n/\log n)$ *provided the floor δ is held fixed rather than tied to m* . The implementation ties them, $\delta = 0.5/(m+1)$, so growing m there would send $\delta \rightarrow 0$, and with it the uniform weight bound $1/\delta$ of Lemma 6 that makes the Lindeberg step trivial and fixes the constants in the law of large numbers; the rate conditions would have to be redone. Decoupling the two is a one-line change to the post-hoc pass and costs nothing (Appendix B, Remark 2), at a retroactive pass of $\mathcal{O}(n m_n k)$ instead of $\mathcal{O}(nk)$; the literal cumulative mixture is the endpoint $m_n = n$ at $\mathcal{O}(n^2 k)$. The measurements above locate the practical knee: on the Gaussian-mean history the reported posterior is within 3×10^{-4} in total variation of the exact n -point mixture by $m = 50$, so a $2.5\times$ more expensive post-hoc pass removes factor (b) almost entirely — which is not the same as achieving $r \equiv 1$, the other three factors being untouched. What the paper reports is the $m = 21$ end, and §5 gives its cost.

Appendix B Proofs

B.1 Setting

We work with a filtration $(\mathcal{F}_i)$ for which, writing $\mathbb{E}_{i-1}[\cdot] = \mathbb{E}[\cdot \mid \mathcal{F}_{i-1}]$, the pair $\xi_i = (\theta_i, \rho_i)$ is $\mathcal{F}_i$ -measurable and the two conditional laws below hold. Particles are indexed by *proposal* time, not arrival time. This is what makes the martingale structure survive asynchronous execution: a worker forms its proposal from the history it has received at that moment, which is part of $\mathcal{F}_{i-1}$, so the conditional density $\tilde{q}_i$ of the emitted candidate is $\mathcal{F}_{i-1}$ -measurable and

$$\theta_i \mid \mathcal{F}_{i-1} \sim \tilde{q}_i, \quad \rho_i \mid \mathcal{F}_{i-1} \vee \sigma(\theta_i) \sim p(\cdot \mid \theta_i). \quad (\text{B4})$$

One further mismatch deserves naming, since it is between the theory and the software rather than inside either. The proofs index by *proposal* time, whereas the implementation’s post-hoc estimator consumes a rank’s *arrival*-ordered log and reconstructs the q_{τ_s} from prefixes of it. The two orderings agree for a sequential run and need not

agree otherwise, so for an asynchronous run the object the code computes is the estimator of (8) evaluated on a different, equally valid history; we do not prove that the theorems transfer between the two, and the discrepancy is one more contributor to r . Under sequential execution $\mathcal{F}_i = \sigma(\xi_1, \dots, \xi_i)$ serves, and (B4) holds by construction. Under *asynchronous* execution it is a genuine assumption, and worth stating precisely what is being assumed. A discrepancy ρ_i may still be in flight when candidate $i + 1$ is proposed, so the σ -field of everything *realized* by that moment does not contain ξ_i , and Lemma 7 and the martingale arrays cannot use it. One must instead take a filtration large enough to contain the in-flight outcomes. Assumption 4 asserts that such a filtration exists *with $\tilde{q}_i$ still the conditional law of the candidate under it*. That is the substantive clause: a worker forms its proposal from the history it has received, so the proposal density is measurable with respect to the smaller field, but conditioning on the larger one need not leave the candidate's law unchanged when completion times steer which rank proposes next. When it does change, $\tilde{q}_i$ should be read as the enlarged-field conditional law rather than as the propagator's emitted density, and (9) is a statement about that object. This is the completion-time selection of §9, seen from the theory side, and we do not prove it away. Concurrency also affects the reconstruction of the q_{τ_s} from a single rank's arrival-ordered log, which enters only through $\|\bar{q}_n - \bar{q}_\infty\|_\infty$ in Assumption 5.

No structural form is imposed on $\tilde{q}_i$: it is whatever the archive rule, the capped rejection, the underflow redraws and the bootstrap phase jointly produce, and it enters the proofs only through $\bar{q}_n^* = \frac{1}{n} \sum_{i \leq n} \tilde{q}_i$ of (6), for which the reported denominator $\bar{q}_n$ of (7) is an m -component stand-in. Let $\mathcal{Q}$ be the set of positive functions of the form (7) whose components satisfy Assumption 2; nothing below requires an element of $\mathcal{Q}$ to integrate to one. Elements of $\mathcal{Q}$ are parameterized by $p_q = (\text{means, component weights, covariances}, \alpha)$ ranging over a compact set $P_q \subset \mathbb{R}^D$, $D = m(kd + k + d(d+1)/2) + m$.

Lemma 6 (bounded weights). *Under Assumptions 2–3, $0 \leq W_{i,n} \leq 1/\delta$ for all $i \leq n$ and all n , surely — whatever the sampler does.*

Proof All components of (7) are non-negative, so $\bar{q}_n \geq \delta\pi$ pointwise on Θ . Hence $W_{i,n} = \pi(\theta_i)K_{\epsilon_n}(\rho_i)/\bar{q}_n(\theta_i) \leq K_{\epsilon_n}(\rho_i)/\delta \leq 1/\delta$, the last step by $K_\epsilon \leq 1$ (Assumption 1). The prior cancels exactly, so the bound needs neither a density-ratio condition nor a moment condition. $\square$

Lemma 7 (uniform SLLN for bounded martingale differences). *Let $P \subset \mathbb{R}^D$ be compact and $\{\psi_p\}_{p \in P}$ measurable functions on a measurable space $\mathcal{X}$ with $\sup_p \|\psi_p\|_\infty \leq B$ and $|\psi_p(x) - \psi_{p'}(x)| \leq \Lambda \|p - p'\|$ for every $x \in \mathcal{X}$. Let (ξ_i) be $\mathcal{X}$ -valued with ξ_i $\mathcal{F}_i$ -measurable. Then*

$$\sup_{p \in P} \left| \frac{1}{n} \sum_{i \leq n} (\psi_p(\xi_i) - \mathbb{E}_{i-1} \psi_p(\xi_i)) \right| \longrightarrow 0 \quad a.s.$$

Proof Fix p . The variables $D_i^p := \psi_p(\xi_i) - \mathbb{E}_{i-1}\psi_p(\xi_i)$ form a martingale difference sequence with $|D_i^p| \leq 2B$, so by the Azuma–Hoeffding inequality $\mathbb{P}(|\sum_{i \leq n} D_i^p| \geq nt) \leq 2\exp(-nt^2/8B^2)$ for every $t > 0$. Fix $\eta > 0$ and let P_η be a finite η -net of P , of cardinality at most $C_P\eta^{-D}$. A union bound gives $\mathbb{P}(\max_{p \in P_\eta} |\sum_{i \leq n} D_i^p| \geq nt) \leq 2C_P\eta^{-D} \exp(-nt^2/8B^2)$, which is summable in n (the net is fixed, so its cardinality does not grow with n); by Borel–Cantelli, $\limsup_n \max_{p \in P_\eta} |\frac{1}{n} \sum_{i \leq n} D_i^p| \leq t$ a.s. For arbitrary $p \in P$ pick $p' \in P_\eta$ with $\|p - p'\| \leq \eta$; then $|D_i^p - D_i^{p'}| \leq 2\Lambda\eta$ by the Lipschitz hypothesis (which also bounds the conditional expectations), so $\limsup_n \sup_{p \in P} |\frac{1}{n} \sum_{i \leq n} D_i^p| \leq t + 2\Lambda\eta$ a.s. Intersecting the almost-sure events over the countable family $t = 1/r$, $\eta = 1/j$ with $r, j \in \mathbb{N}$, gives the claim. $\square$

Lemma 8 (conditional-mean identity). *Let $\phi : \Theta \rightarrow \mathbb{R}$ be bounded measurable, let $\ell \in \{1, 2\}$ (an exponent, not the fidelity ratio r), and let $\epsilon \in [a, \epsilon_0]$ for some $a > 0$ and the positive function $q \geq \delta\pi$ be either deterministic or $\mathcal{F}_{i-1}$ -measurable. Then under (B4), writing $L_\epsilon^{(1)} := L_\epsilon$,*

$$\mathbb{E}_{i-1} \left[\frac{\phi(\theta_i) K_\epsilon(\rho_i)^\ell}{q(\theta_i)^\ell} \right] = \int_\Theta \phi L_\epsilon^{(\ell)} \frac{\tilde{q}_i}{q^\ell} d\theta.$$

Proof Condition on $\mathcal{F}_{i-1}$, given which ϵ and q are constants, and integrate first over $\rho_i \sim p(\cdot \mid \theta_i)$, which gives the factor $L_\epsilon^{(\ell)}(\theta_i) = \mathbb{E}[K_\epsilon(\rho)^\ell \mid \theta_i]$, then over $\theta_i \sim \tilde{q}_i$. The $\mathcal{F}_{i-1}$ -measurable case is what Steps 2 and 3 of the proof of Theorem 3 need; an $\mathcal{F}_\infty$ -measurable q — such as the limit $\bar{q}_\infty$ — would *not* be admissible here, which is why that proof centres on predictable proxies rather than on the limit. Fubini applies because the integrand is bounded by $\|\phi\|_\infty/(\delta\pi_-)^\ell$. The case $\ell = 1$, $\phi = \pi h$ is the one used for first moments; $\ell = 2$, $\phi = \pi^2 h^2$ gives the conditional second moments of Step 3(b) and Corollary 4. $\square$

The class used below.

Take $\mathcal{X} = \Theta \times [0, \infty)$, $\xi_i = (\theta_i, \rho_i)$, and, for a fixed bounded measurable h ,

$$\psi_{(\epsilon, p_q)}(\theta, \rho) := \frac{\pi(\theta) h(\theta) K_\epsilon(\rho)}{q_{p_q}(\theta)}, \quad (\epsilon, p_q) \in P_a := [a, \epsilon_0] \times P_q, \quad a \in (0, \epsilon_0]. \quad (\text{B5})$$

Each P_a is compact *and deterministic*, which matters: Lemma 7 cannot be applied over a parameter set that depends on the run, and $[\epsilon_\infty, \epsilon_0]$ would be such a set when ϵ_∞ is random. We therefore apply the lemma on $P_{1/j}$ for every $j \in \mathbb{N}$ with $1/j \leq \epsilon_0$ and intersect the countably many almost-sure events; since $\epsilon_n \downarrow \epsilon_\infty > 0$ a.s., each path has $\epsilon_n \in [1/j, \epsilon_0]$ for all n once $1/j \leq \epsilon_\infty$, so the resulting uniform bound covers $(\epsilon_n, \bar{q}_n)$ on that path. No deterministic bandwidth floor is required. On P_a , $\sup \|\psi\|_\infty \leq \|h\|_\infty/\delta$ by the computation of Lemma 6. For the Lipschitz hypothesis: each truncated component density $\varphi(\theta; \mu_j, \Sigma)/Z(\mu_j, \Sigma)$ has parameter-derivatives bounded uniformly over $\theta \in \Theta$, because Θ is compact, $\sigma_-^2 I \preceq \Sigma \preceq \sigma_+^2 I$ bounds φ and its derivatives, and the truncation mass is bounded below, $Z(\mu_j, \Sigma) \geq c_0 > 0$, uniformly for $\mu_j \in \Theta$. Here it matters that the implementation’s constant is the *product of per-axis in-box masses*, not the exact joint mass: each factor is $\Phi((\text{hi}_d - \mu_{jd})/\sigma_d) - \Phi((\text{lo}_d - \mu_{jd})/\sigma_d)$, and since

μ_{jd} lies in $[\text{lo}_d, \text{hi}_d]$ that interval contains a half-width $w_d/2$ on one side of μ_{jd} , giving a factor at least $\Phi(w_d/2\sigma_+) - \frac{1}{2} > 0$; the product over coordinates, $\prod_d (\Phi(w_d/2\sigma_+) - \frac{1}{2})$, is therefore bounded below by a constant depending only on the box and σ_+ , and is smooth in (μ_j, Σ) with bounded derivatives there. This is where Assumption 2's box hypothesis is used; compactness of Θ alone would not give it. Hence every $q \in \mathcal{Q}$ — a set of positive *functions*, not necessarily normalized (Assumption 2) — obeys $\delta\pi_- \leq q \leq q_+ < \infty$ and $q \mapsto q(\theta)$ is Lipschitz in p_q uniformly in θ ; so is $1/q$, since q is bounded away from 0. With K_ϵ Lipschitz in ϵ (Assumption 1) and πh bounded, (B5) satisfies the hypotheses of Lemma 7. Note that on $\{1/j \leq \epsilon_\infty\}$ we have $(\epsilon_n, \bar{q}_n) \in P_{1/j}$ for every n , and that $\bar{q}_\infty \geq \delta\pi$ by taking uniform limits.

B.2 Proof of Theorem 1

Work on the event $\{1/j \leq \epsilon_\infty\}$ for a fixed j , so that $\hat{p}_n := (\epsilon_n, \bar{q}_n) \in P_{1/j}$ for every n ; the conclusion then follows on the union over j , which has probability one. Fix a bounded measurable h and define, for each *fixed* $p \in P_{1/j}$, $\Gamma_i(p) := \mathbb{E}_{i-1}[\psi_p(\xi_i)]$ and $\bar{\Gamma}_n(p) := \frac{1}{n} \sum_{i \leq n} \Gamma_i(p)$. On the almost-sure event of Lemma 7 for $P_{1/j}$ the bound holds simultaneously for all $p \in P_{1/j}$, so the *function* $\bar{\Gamma}_n$ may be evaluated at the random $\hat{p}_n$:

$$\frac{1}{n} \sum_{i \leq n} W_{i,n} h(\theta_i) = \bar{\Gamma}_n(\hat{p}_n) + o(1) \stackrel{\text{Lem. 8}}{=} \int_{\Theta} \pi h L_{\epsilon_n} \frac{\bar{q}_n^*}{\bar{q}_n} d\theta + o(1) \quad \text{a.s.}, \quad (\text{B6})$$

using $\frac{1}{n} \sum_{i \leq n} \tilde{q}_i = \bar{q}_n^*$. The middle term is the function $\bar{\Gamma}_n(\cdot)$ evaluated at $\hat{p}_n$ and must not be read as $\mathbb{E}_{i-1}[\psi_{\hat{p}_n}(\xi_i)]$: $\hat{p}_n$ depends on draws after i , is not $\mathcal{F}_{i-1}$ -measurable, and that conditional expectation is not what Lemma 7 controls. Split the integral against $\int \pi h L_{\epsilon_\infty}$:

$$\left| \int \pi h L_{\epsilon_n} \frac{\bar{q}_n^*}{\bar{q}_n} - \int \pi h L_{\epsilon_\infty} r \right| \leq \|h\|_\infty \pi_+ |\Theta| \left[R L_K(1/j) |\epsilon_n - \epsilon_\infty| + \left\| \frac{\bar{q}_n^*}{\bar{q}_n} - r \right\|_\infty \right],$$

on the event $\{1/j \leq \epsilon_\infty\}$ of the localization above, using $|L_{\epsilon_n} - L_{\epsilon_\infty}| \leq L_K(1/j) |\epsilon_n - \epsilon_\infty|$ (Assumption 1 and Jensen; every ϵ_n and ϵ_∞ lies in $[1/j, \epsilon_0]$ there) and $L_{\epsilon_\infty} \leq 1$, and where $R < \infty$ bounds $\bar{q}_n^*/\bar{q}_n$ for all large n — not from any structural bound on $\tilde{q}_i$, of which we assume none, but because (9) sends that ratio uniformly to $r \leq r_+$. Both bracketed terms vanish, the first because $\epsilon_n \rightarrow \epsilon_\infty$ and the second by (9). Hence $\frac{1}{n} \sum_i W_{i,n} h(\theta_i) \rightarrow \int \pi h L_{\epsilon_\infty} r$ a.s. Applying this with $h = f$ and with $h \equiv 1$ and dividing — legitimate because $Z_r = \int \pi L_{\epsilon_\infty} r \geq r_- Z_{\epsilon_\infty} > 0$ — gives $\hat{\pi}_n(f) \rightarrow \pi_{\epsilon_\infty}^r(f)$ a.s. Corollary 2 is the case $r \equiv 1$. $\square$

Remark 1. Consistency used Assumption 5(i) only through (9) and $\epsilon_n \rightarrow \epsilon_\infty$, and nothing at all about the form of $\tilde{q}_i$. This is what makes the theorem a statement about the algorithm as implemented rather than about an idealized variant of it: any sampler whose draw mixture the reported denominator tracks, in the ratio sense of (9), is covered — the archive rule, the capped rejection and the underflow redraws included. It is also the content that earlier drafts' condition on convergence to the

idealized cumulative mixture was missing: that object is not what the argument needs, and on its own does not deliver it.

B.3 Proof of Theorem 3 and Corollary 4

Let $g := f - \pi_{\epsilon_\infty}^r(f)$, a fixed bounded function — Assumption 5(ii) makes ϵ_∞ and r , hence $\pi_{\epsilon_\infty}^r(f)$, deterministic — satisfying $\int \pi g L_{\epsilon_\infty} r = 0$. Since $\sum_i W_{i,n} g(\theta_i) = (\sum_i W_{i,n})(\hat{\pi}_n(f) - \pi_{\epsilon_\infty}^r(f))$ identically and $\frac{1}{n} \sum_i W_{i,n} \rightarrow Z_r > 0$ a.s. (Theorem 1 with $h \equiv 1$), by Slutsky's theorem — which preserves stable convergence — it suffices to prove

$$S_n := \frac{1}{\sqrt{n}} \sum_{i \leq n} \frac{\pi(\theta_i) g(\theta_i) K_{\epsilon_n}(\rho_i)}{\bar{q}_n(\theta_i)} \longrightarrow \mathcal{N}(0, v^2) \text{ stably}, \quad v^2 := \int_{\Theta} \pi^2 g^2 L_{\epsilon_\infty}^{(2)} \frac{\tilde{q}_\infty}{\bar{q}_\infty^2} d\theta, \quad (\text{B7})$$

after which $\sigma_f^2 = v^2 / Z_r^2$, which is (A2).

Step 1 (replacing the parameters by predictable proxies). $\bar{q}_\infty$ and ϵ_∞ are in general only $\mathcal{F}_\infty$ -measurable, so an array built from them is *not* adapted and Lemma 8 does not apply to it. We therefore centre on the predictable pair instead: let $\bar{q}_{i-1} \in \mathcal{Q}$ be the denominator (7) formed from the first $i-1$ draws and ϵ_{i-1} the bandwidth in force at the i -th proposal, both $\mathcal{F}_{i-1}$ -measurable, and put

$$V_i := \frac{\pi(\theta_i) g(\theta_i) K_{\epsilon_{i-1}}(\rho_i)}{\bar{q}_{i-1}(\theta_i)}.$$

Since $\bar{q}_n, \bar{q}_{i-1} \geq \delta\pi_-$ and $K \leq 1$, and $|\epsilon_n - \epsilon_{i-1}| \leq |\epsilon_n - \epsilon_\infty| + |\epsilon_{i-1} - \epsilon_\infty|$ (likewise for $\bar{q}$),

$$\left| S_n - \frac{1}{\sqrt{n}} \sum_{i \leq n} V_i \right| \leq C\sqrt{n}(|\epsilon_n - \epsilon_\infty| + \|\bar{q}_n - \bar{q}_\infty\|_\infty) + \frac{C}{\sqrt{n}} \sum_{i \leq n} (|\epsilon_{i-1} - \epsilon_\infty| + \|\bar{q}_{i-1} - \bar{q}_\infty\|_\infty)$$

with $C = \|g\|_\infty \pi_+ \max\{L_K(\epsilon_\infty)/(\delta\pi_-), (\delta\pi_-)^{-2}\}$ — a single constant here, since Assumption 5(ii) makes ϵ_∞ deterministic, so the central limit theorem needs no localization, and both brackets vanish by (10). This step is what the rates buy; the remainder of the proof never touches $\bar{q}_n$ or ϵ_n again.

Step 2 (drift). Lemma 8 with $\ell = 1$, $\phi = \pi g$ applies to V_i because $(\epsilon_{i-1}, \bar{q}_{i-1})$ is $\mathcal{F}_{i-1}$ -measurable and g is a fixed function under the central-limit clause of Assumption 5; with $\int \pi g L_{\epsilon_\infty} r = 0$,

$$c_i := \mathbb{E}_{i-1} V_i = \int \pi g \left(L_{\epsilon_{i-1}} \frac{\tilde{q}_i}{\bar{q}_{i-1}} - L_{\epsilon_\infty} r \right) d\theta,$$

$$|c_i| \leq C'(|\epsilon_{i-1} - \epsilon_\infty| + \|\tilde{q}_i - \tilde{q}_\infty\|_\infty + \|\bar{q}_{i-1} - \bar{q}_\infty\|_\infty),$$

C' depending only on $\|g\|_\infty, \pi_\pm, \delta, |\Theta|, L_K(\epsilon_\infty)$. The second half of (10) is exactly the statement that $n^{-1/2} \sum_{i \leq n}$ of that bound tends to zero, so $n^{-1/2} \sum_{i \leq n} c_i \rightarrow 0$. (Under the sufficient condition that each of the three terms is $O(i^{-1/2-a})$ this holds in each of

the cases $a < 1/2$, $a = 1/2$, $a > 1/2$, where the partial sums are $O(n^{1/2-a})$, $O(\log n)$ and $O(1)$ respectively.)

Step 3 (martingale CLT). Let $X_{n,i} := n^{-1/2}(V_i - c_i)$, a martingale difference array with respect to the filtration $(\mathcal{F}_i)$, which does not depend on n and is therefore nested. Two conditions: (a) *Conditional Lindeberg.* $|X_{n,i}| \leq 2\|g\|_\infty \pi_+ / (\delta \pi_- \sqrt{n})$ surely, so for every $\eta > 0$ the truncated sum $\sum_i \mathbb{E}_{i-1}[X_{n,i}^2 \mathbf{1}\{|X_{n,i}| > \eta\}]$ is exactly zero for all n large. The defensive floor makes this step trivial where an unbounded-weight scheme would need a uniform-integrability argument. (b) *Conditional variance.* Lemma 8 with $\ell = 2$, $\phi = \pi^2 g^2$ gives

$$\mathbb{E}_{i-1} V_i^2 = \int \frac{\pi^2 g^2 L_{\epsilon_{i-1}}^{(2)}}{\bar{q}_{i-1}^2} \tilde{q}_i d\theta = v^2 + R_i,$$

$$v^2 = \int \pi^2 g^2 L_{\epsilon_\infty}^{(2)} \frac{\tilde{q}_\infty}{\bar{q}_\infty^2} d\theta, \quad |R_i| \leq C''(|\epsilon_{i-1} - \epsilon_\infty| + \|\tilde{q}_i - \tilde{q}_\infty\|_\infty + \|\bar{q}_{i-1} - \bar{q}_\infty\|_\infty),$$

where C'' carries $2L_K(\epsilon_\infty)$ for the squared kernel, since $|K_\epsilon^2 - K_{\epsilon'}^2| = |K_\epsilon - K_{\epsilon'}|(|K_\epsilon + K_{\epsilon'}|) \leq 2L_K(\epsilon_\infty)|\epsilon - \epsilon'|$. The bound on $|R_i|$ has vanishing Cesàro average, a fortiori from (10), so $\frac{1}{n} \sum_{i \leq n} \mathbb{E}_{i-1} V_i^2 \rightarrow v^2$ a.s.; with $\frac{1}{n} \sum_i c_i^2 \rightarrow 0$ from Step 2, $\sum_i \mathbb{E}_{i-1} X_{n,i}^2 = \frac{1}{n} \sum_i (\mathbb{E}_{i-1} V_i^2 - c_i^2) \rightarrow v^2$ a.s. Note that v^2 is permitted to be random here — an $\mathcal{F}_\infty$ -measurable limit of the conditional variances is exactly the setting the martingale CLT below covers, and it is only inside $\mathbb{E}_{i-1}$ that predictability was needed. The martingale central limit theorem for arrays with nested σ -fields, in its conditional-variance form [26, Cor. 3.1, pp. 58–59], then gives $\sum_i X_{n,i} \rightarrow \mathcal{N}(0, v^2)$ stably; the convergence is stable rather than merely distributional precisely because v^2 is $\mathcal{F}_\infty$ -measurable and possibly random. Combining Steps 1–3 gives (B7). $\square$

Proof of Corollary 4 Write $\hat{\sigma}_n^2 = [\frac{1}{n} \sum_i W_{i,n}^2 (f(\theta_i) - \hat{\pi}_n(f))^2] / [\frac{1}{n} \sum_i W_{i,n}]^2$. In the numerator, $(f - \hat{\pi}_n(f))^2 = g^2 + 2g(\pi_{\epsilon_\infty}^r(f) - \hat{\pi}_n(f)) + (\pi_{\epsilon_\infty}^r(f) - \hat{\pi}_n(f))^2$, and $W_{i,n} \leq 1/\delta$ with $\hat{\pi}_n(f) \rightarrow \pi_{\epsilon_\infty}^r(f)$ a.s. (Theorem 1), so the cross terms vanish and it suffices to treat $\frac{1}{n} \sum_i W_{i,n}^2 g(\theta_i)^2$. Apply Lemma 7 on P_a with $a = \epsilon_\infty$ — deterministic here, so no localization is needed — to the class $\psi_{(\epsilon, p_q)}(\theta, \rho) = \pi(\theta)^2 g(\theta)^2 K_\epsilon(\rho)^2 / q_{p_q}(\theta)^2$, which is bounded by $\|g\|_\infty^2 / \delta^2$ and Lipschitz in (ϵ, p_q) by the argument of (B5) with $2L_K(\epsilon_\infty)$ in place of L_K ; the substitution of $\hat{p}_n$ used in (B6), followed by Lemma 8 with $\ell = 2$, $\phi = \pi^2 g^2$ (a fixed function under Assumption 5(ii)), gives $\frac{1}{n} \sum_i W_{i,n}^2 g^2(\theta_i) = \int \pi^2 g^2 L_{\epsilon_n}^{(2)} (\bar{q}_n^* / \bar{q}_n) \bar{q}_n^{-1} + o(1)$, and $\epsilon_n \rightarrow \epsilon_\infty$, $\|\bar{q}_n^* / \bar{q}_n - r\|_\infty \rightarrow 0$ and $\|\bar{q}_n - \bar{q}_\infty\|_\infty \rightarrow 0$ (the last from (10)) send this to $\int \pi^2 g^2 L_{\epsilon_\infty}^{(2)} r / \bar{q}_\infty = \int \pi^2 g^2 L_{\epsilon_\infty}^{(2)} \tilde{q}_\infty / \bar{q}_\infty^2 = v^2$ a.s., using $r = \tilde{q}_\infty / \bar{q}_\infty$. The denominator tends to Z_r^2 , so $\hat{\sigma}_n^2 \rightarrow v^2 / Z_r^2 = \sigma_f^2$ a.s. Because the convergence in Theorem 3 is stable, the pair $(\sqrt{n}(\hat{\pi}_n(f) - \pi_{\epsilon_\infty}^r(f)), \hat{\sigma}_n)$ converges jointly to $(\sigma_f \mathcal{Z}, \sigma_f)$ — stable convergence is exactly joint convergence with every $\mathcal{F}_\infty$ -measurable variable, and σ_f is one — so with $\sigma_f > 0$ a.s. the continuous mapping theorem gives $\mathcal{Z} \sim \mathcal{N}(0, 1)$ as the limit of the ratio. Marginal convergence to a mixed normal alone would not suffice. $\square$

Remark 2 (growing snapshot count). Nothing above requires m to be fixed except the compactness and fixed dimension D of P_q — and the fixed floor δ , which the implementation ties to m as $0.5/(m+1)$ and which must be decoupled before m is

allowed to grow, since Lemma 6’s bound is $1/\delta$. With δ fixed and $m = m_n \rightarrow \infty$, run the proof of Lemma 7 with an n -dependent net: the union bound costs $D_n \log(1/\eta_n)$ with $D_n = \Theta(m_n)$, and the Lipschitz constant over the mixture weights grows at most polynomially in m_n , so choosing $\eta_n \rightarrow 0$ fast enough leaves the Borel–Cantelli step summable for any $m_n = o(n/\log n)$. In that regime the snapshot clause of Assumption 5 follows from $V_n/m_n \rightarrow 0$, with $V_n = \sum_{i < n} \|q_{i+1} - q_i\|_\infty$ the total variation of the proposal path, rather than having to be assumed: the draw-proportional m_n -point mixture is then a converging quadrature of the empirical mixture. The retroactive pass costs $\mathcal{O}(n m_n k)$.

B.4 Proof of Proposition 5

Write $\zeta_0 := \pi_{\epsilon_\infty}(|r - 1|)$. Under the hypothesis, $\zeta_0 \leq \eta_A + \pi_{\epsilon_\infty}(A^c) = \zeta$. For bounded f , since $\pi_{\epsilon_\infty}^r(f) = \pi_{\epsilon_\infty}(fr)/\pi_{\epsilon_\infty}(r)$,

$$\pi_{\epsilon_\infty}^r(f) - \pi_{\epsilon_\infty}(f) = \frac{\pi_{\epsilon_\infty}(f(r - 1)) - \pi_{\epsilon_\infty}(f) \pi_{\epsilon_\infty}(r - 1)}{\pi_{\epsilon_\infty}(r)},$$

whose modulus is at most $2\|f\|_\infty \zeta_0/(1 - \zeta_0)$ because $\pi_{\epsilon_\infty}(r) \geq 1 - \zeta_0$. Taking the supremum over $\|f\|_\infty \leq 1$ and halving bounds the total variation by $\zeta_0/(1 - \zeta_0)$; as $x \mapsto x/(1 - x)$ increases on $[0, 1)$, this is at most $\zeta/(1 - \zeta)$ whenever $\zeta < 1$, which is (A3). For $\zeta \geq 1$ the bound degenerates and only the trivial $\|\cdot\|_{\text{TV}} \leq 1$ survives.

The closed form quoted in §A.1(a) for the prior floor’s contribution follows by evaluating (9) for that contribution alone: with $u = q_\infty/\pi$ and a limiting bootstrap share $\nu_\infty < \delta$,

$$r_{\text{floor}} - 1 = \frac{\nu_\infty + (1 - \nu_\infty)u}{\delta + (1 - \delta)u} - 1 = \frac{(\delta - \nu_\infty)(u - 1)}{\delta + (1 - \delta)u},$$

so on $\{u \geq c\}$, splitting at $u = 1$ and bounding each branch, $|r_{\text{floor}} - 1| \leq (\delta - \nu_\infty) \max\{(1 - \delta)^{-1}, (\delta + (1 - \delta)c)^{-1}\}$, which is the η_A of Proposition 5 for $A = \{u \geq c\}$. It is identically 1 when $\nu_\infty \geq \delta$, and $\rightarrow 1$ as $\delta \rightarrow 0$. $\square$

B.5 Status of each assumption in the implementation

Appendix C Implementation Details

Tolerance schedulers.

Every scheduler is a pure function of the evaluated history and is *monotone by construction*: the propagator clips any proposed bandwidth above the history-reconstructed running minimum ϵ_{hist} (§3.2), so the effective tolerance can only decrease. Let μ denote the extra-individual margin (default $\mu = k$); it is unrelated to the snapshot count m of §4. The three base rules act on the individuals *within* the current bandwidth, i.e. the hard-threshold set $\{i : \rho_i < \epsilon\}$. (This bandwidth-membership count is used *only* to drive the tolerance schedule; it is distinct from the smooth-kernel weight $K_\epsilon(\rho_i)$ that defines the proposal mixture and the reported posterior, for which

every evaluated particle contributes continuously. We avoid the word “accepted” here precisely because acceptance is probabilistic under a smooth kernel.):

- **Quantile.** $\epsilon \leftarrow$ the lower-rank (non-interpolated) p -th percentile of the accepted losses, default $p = 50$; applied only once at least $k + m$ individuals are accepted.
- **Geometric decay.** $\epsilon \leftarrow \epsilon_0 \gamma^e$ after each completed epoch e of $k + \mu$ accepted individuals, default $\gamma = 0.9$.
- **Acceptance rate.** Over a sliding window of the most recent $k + \mu$ individuals, if the acceptance rate exceeds r_{hi} (default 0.3) tighten by a factor 0.9, otherwise hold; the rule never expands ϵ (the low-rate guard $r_{\text{lo}} = 0.1$ simply holds).

With a smooth (Gaussian or Epanechnikov) kernel the schedulers run in a *kernel-aware* mode: instead of the hard $\rho < \epsilon$ count, ϵ is chosen by bisection so that the per-step kernel-weighted effective-sample-size retention ratio meets a target (default 0.95), which keeps the bandwidth schedule comparable across kernel choices.

Proposal covariance.

The archive A_n is the top- k history members by smallest discrepancy. Component weights are formed in log-space as $\tilde{W}_j \propto w_j K_{\epsilon_n}(\rho_j)$ and normalised (a uniform fallback is used if all smooth weights underflow). The perturbation covariance is $\Sigma_n = s [\widehat{\text{Cov}}_{\tilde{W}}(A_n) + \lambda_n I]$ with s the `perturbation_scale` and a two-part jitter $\lambda_n = 10^{-9} \text{tr } \widehat{\text{Cov}}_{\tilde{W}}(A_n)/d + 10^{-12} \bar{b}^2$, where $\bar{b}^2$ is the mean squared prior-box width; it is Cholesky-factored once per call. The first term is scale-aware, so a tightly converged archive (marginal standard deviation $\sim 10^{-4}$) is not swamped by the floor as a fixed 10^{-6} would swamp it; the second is a run-independent absolute floor that keeps the matrix factorable for a degenerate archive and supplies the uniform lower spectral bound Assumption 2 needs (Table B1). Each mixture component is truncated to the prior box, with its log normalising mass computed as the product of the per-axis in-box masses. That is exact for a diagonal covariance, in which case the proposal integrates to one on the support; for a correlated covariance it is an approximation, so the evaluated proposal is a positive function that need not integrate to one — which is why §4 states its denominator assumption for positive functions and books the discrepancy as factor (c) of r (§A.1).

Matched pyABC acceptor.

For the synchronous baseline, pyABC’s `UniformAcceptor` is replaced, for smooth kernels, by a probabilistic-rejection acceptor that accepts a candidate of discrepancy ρ with probability $K_\epsilon(\rho)$ (peak-normalised so $K_\epsilon(0) = 1$), using the *same* kernel K_ϵ as the asynchronous side; the hard kernel falls back to `UniformAcceptor`. The rejection step of each replicate draws from an independently seeded generator. This is the sense in which the kernel and acceptor are matched across methods (§5).

Bounded-memory posterior estimator.

The retroactive AMIS estimator (8) evaluates the snapshot denominator $\bar{q}_n$ — a draw-proportional mixture of $m \leq S+1 = 21$ history-reconstructed proposals with

a defensive prior component (mass floored at $0.5/(m+1)$) — at each of the n evaluated particles, at total cost $\mathcal{O}(nmk) = \mathcal{O}(nk)$ with m fixed. The snapshots are taken draw-proportionally from the reconstructed archive (indices $0, \text{step}, 2\text{step}, \dots$ with $\text{step} = \max(1, \lfloor |\text{archive}|/S \rfloor)$), plus the most recent archive-phase proposal if it is not already among them — hence $m \leq S+1 = 21$ rather than S . For near-instantaneous simulators the history reaches $\mathcal{O}(10^7)$ particles, so the pass is accumulated in chunks of $2^{16} = 65,536$ history points: peak memory is $\mathcal{O}(\text{chunk} \cdot m \cdot k)$ rather than $\mathcal{O}(n)$. Each particle’s denominator is evaluated independently within a single chunk (there is no log-sum-exp reduction *across* chunk boundaries), so chunking changes memory, not the value, and the result is bit-identical to the unchunked computation. Benchmarked directly (`scripts/diag_retrospective_cost.py`, artifact committed) at $k = 100$, $m = 20$ over n from 2.5×10^4 to 10^6 , the pass costs 49.3 s per 10^6 particles on a single core and is linear in n to within a 7.9% maximum relative residual, so the 3×10^7 -particle Gaussian history extrapolates to ≈ 25 minutes (and 10^7 to ≈ 8 minutes). Peak working set grows by 0.27 GB over that $40\times$ range of n — i.e. it is flat in n as the chunking predicts, the intermediate being one 2^{16} -point chunk. The pass parallelizes trivially across evaluation points; it runs after the timed budget and is excluded from the throughput measurements by design. For a near-instantaneous simulator this is comparable to the timed budget itself — non-trivial though bounded — whereas for cost-bearing simulators, whose histories are orders of magnitude shorter, it is negligible against the simulation time.

Appendix D Additional Experimental Protocol

Effective simulating ranks.

A “ W -worker” point allocates W physical cores to both arms, but the two arms use them differently. Every rank of the asynchronous method evaluates, so it has W simulating ranks. pyABC’s MPI sampler is root-driven: rank 0 distributes work to ranks $1, \dots, W-1$ and does not simulate, so the baseline has $W-1$. We report the allocated core count W for both, which flatters the asynchronous arm by $1/W$ — 2.1% at $W=48$, 0.3% at $W=384$. We do not correct for it because the correction shrinks monotonically with scale while the gap we report grows with scale, so it cannot account for any reported effect; but it should be read as part of the coordination cost the barrier-free design avoids rather than as a measurement we have netted out.

Seeding.

Each experiment derives its replicate seeds deterministically from a single base seed via `make_seeds`: a `random.Random(base)` stream emits unique non-negative 31-bit integers, one per replicate, portable across platforms and independent of any global RNG state. Derived seeds (observed-data generation, the pyABC rejection step) use a structured BLAKE2b hash (`stable_seed`) of their inputs, so identical inputs reproduce identical seeds across fresh and resumed (`--extend`) runs.

Crash recovery (kill-and-resume).

We test the crash-recoverability claim of §3.1 directly. On the Gaussian-mean benchmark (4 workers, a 1.2×10^5 -simulation budget) we compare a clean uninterrupted run against a run that is hard-killed (SIGKILL of the entire worker process group) partway through inference and then relaunched with an identical command; recovery reloads the periodic population checkpoint and resumes. The resumed run recovers the analytic posterior mean — absolute error 7.6×10^{-3} , against 7.5×10^{-4} for the clean run — and completes the same total simulation budget with no duplicated records. It is *not* a bit-identical replay: the two reported posterior means differ by 6.9×10^{-3} . Three things prevent the replay, and they are worth separating. Two are incidental — the candidate generator is re-seeded from its (replicate, rank) key on restart rather than checkpointed, and MPI arrival order is not deterministic. The third is structural: the online AMIS snapshot buffer is rebuilt empty on restart, and since it enters the importance denominator of every subsequent candidate, and those weights form the archive’s proposal mixture, the post-restart proposal sequence is drawn from a different (though equally valid) adaptation trajectory (§3.1). Checkpointing the buffer would remove the third but not the first two. This is the distinction §3.1 draws between the estimator and the running sampler: recovery yields a *valid* posterior trajectory over the actual evaluated history, not the identical one. In this instance the kill landed *during* a checkpoint write and truncated the primary checkpoint; recovery fell back to the previous-state backup that the dumper writes before overwriting, so the resume succeeded automatically — the reported estimator, being a pure function of the recovered history, is unaffected by which of the two checkpoints is used.

Cluster and wall-clock control.

Experiments run on JUWELS (Forschungszentrum Jülich) under ParaStation MPI, 48 cores per node, scheduled with SLURM. All wall-clock-budgeted methods stop at a fixed wall-clock budget enforced uniformly by a monotonic-clock deadline with first-rank-hit semantics: once any rank trips the deadline it serialises its current state into the records and returns, so a partial final generation is recorded rather than discarded (the simulation-limited studies marked † in Table D2 — the SBC and kill-and-resume runs — instead stop at a fixed per-trial simulation count). Posterior estimation and plotting run off the timed inference path, so they do not count against the budget.

Benchmark configurations.

Table D2 lists the resolved settings (k is the archive/population size; the budget is per method run). The strong-scaling studies sweep the worker count from 1 to 288 (Lotka–Volterra; sub-node 1/4/16 and fully-packed multiples of 48 up to six nodes) and to 384 (Cellular Potts; up to eight nodes), with archive size $k \in \{100, 1000\}$; all other studies fix the workers shown.

Benchmark-specific notes.

Each benchmark fixes one observed dataset (generated from seed 42) and infers parameters under uniform priors; the discrepancy is a distance between simulated and observed summary statistics. *Gaussian mean*: $n_{\text{obs}} = 100$ observations $\sim \mathcal{N}(0, 1)$,

prior $\mu \sim \mathcal{U}[-5, 5]$, summary the sample mean, discrepancy $|\bar{x}_{\text{sim}} - \bar{x}_{\text{obs}}|$; the closed-form Gaussian posterior is the calibration and recovery reference. *g-and-k*: the quantile-defined g-and-k distribution ($c = 0.8$), $n_{\text{obs}} = 1000$, true $(A, B, g, k) = (3, 1, 2, 0.5)$, priors $A \sim \mathcal{U}[0, 6]$, $B \sim \mathcal{U}[0.1, 4]$, $g \sim \mathcal{U}[0, 5]$, $k \sim \mathcal{U}[0, 1]$; summary the seven octiles (the 0.125, ..., 0.875 quantiles), discrepancy Euclidean. *Lotka-Volterra*: a Gillespie stochastic simulation of the four-reaction predator-prey system from $(X_0, Y_0) = (50, 100)$ over $T = 30$, true rates $\theta = (0.5, 0.025, 0.025, 0.5)$, priors $\theta_1, \theta_4 \sim \mathcal{U}[0.05, 2]$ and $\theta_2, \theta_3 \sim \mathcal{U}[0.001, 0.2]$; summary the six statistics $(\bar{X}, \text{sd } X, \bar{Y}, \text{sd } Y, \log(1 + \text{var } X/\bar{X}), \log(1 + \text{var } Y/\bar{Y}))$, discrepancy Euclidean. *Cellular Potts*: the real `cellsInSilico` (NASTJA) simulator, inferring the cell division rate and the target cell volume under log-uniform priors on $[0.001, 0.2]$ and $[200, 1200]$ (truths 0.009 and 500); the discrepancy is the equally weighted distance over the log cell count and the log 95th-percentile radius, each averaged over four replicate simulations, against the same summaries of the synthetic observation; the setup and the screening that fixed it are in Appendix E. Its per-evaluation cost dominates, which is why it uses the longest wall-clock budget.

Computing environment.

All experiments ran on the JUWELS Cluster at the Jülich Supercomputing Centre: dual-socket Intel Xeon Platinum 8168 (Skylake) nodes with 48 physical cores and ≈ 94 GB memory per node, connected by Mellanox EDR InfiniBand. The production campaign of the three cheap benchmarks (Gaussian mean, g-and-k, Lotka-Volterra) instead placed its 48 ranks as 12 per node on four nodes after the single-node submissions failed; the Cellular Potts production run, and the straggler, heterogeneity, twin and scaling campaigns, are single-node or fully packed. The software stack is ParaStationMPI 5.10 (the 2025 GCC toolchain), Python 3.12.3, NumPy 1.26.4, SciPy 1.13.1, mpi4py 4.0.1, and pyABC 0.12.17, with POT for the sliced Wasserstein distance; the asynchronous method runs as a propagator in our fork of Propulate. Unless noted, ranks are placed one per physical core (`srun --ntasks-per-node=48 --cpus-per-task=1`), so a “ W -worker” point occupies W physical cores packed onto $\lceil W/48 \rceil$ fully-reserved nodes; the cheap-benchmark production campaign instead placed 12 ranks per node on four nodes (`--nodes=4 --ntasks-per-node=12`) under whole-node reservation, which is why its asynchronous throughput is below the packed single-node scaling run’s on the same benchmark (§7.3). Each strong-scaling point reports five replicates (three to five for the largest Cellular Potts points), and figures show medians with inter-quartile ranges.

Appendix E The Cellular Potts inference setup

The benchmark runs the `cellsInSilico` Cellular Potts tissue model [25] through the NASTJA framework: a 50^3 lattice for 501 steps (an 80^3 lattice for 1001 steps in the larger configuration), seeded with a small cluster of cells that divide and move, summarised at the final step. The inference problem was designed on a screening campaign of about 12,000 simulations over space-filling designs of six candidate parameters,

scored by the ratio of between-parameter to within-parameter variation of each summary block. Three findings fixed the setup. Of the seven summary blocks the shipped configuration weighted equally, two scalars — the log cell count and the log 95th-percentile radius — carry essentially all of the signal (signal-to-noise 12 and 9 against about 1 for the four ten-dimensional curve blocks), so the discrepancy is taken over those two, equally weighted. One evaluation is the feature-average of four replicate simulations at the same parameters, because a single realisation is too noisy for the discrepancy to order nearby parameters. And the two summaries resolve exactly two directions — how many cells there are, and how large the cluster is at a given count — so the inferred parameters are the two that move them independently: the division rate (count) and the target cell volume (radius at fixed count), whose responses are near-orthogonal ($|\cos| = 0.07$). Every other candidate expresses itself through the count: cell motility, adhesion and surface tension are confounded with the division rate at $|\cos| \geq 0.8$ at 50^3 , and at 80^3 , with $6.4\times$ the cells, the confounding sharpens rather than breaks ($0.81 \rightarrow 0.98$ for the surface term), which is why the benchmark is two-dimensional as a property of the model rather than for want of screening. Priors are log-uniform on $[0.001, 0.2]$ and $[200, 1200]$ with truths 0.009 and 500; motility is fixed at 1400. Rejection ABC on 2000 prior draws forecast 91% and 64% contraction for the two parameters before any adaptive run, against the 92% and 81% the asynchronous estimator reports (§7.4).

Two configurations.

Two configurations of the benchmark appear in this paper. The *production* configuration is the two-parameter setup above — division rate and target cell volume, four replicate simulations per evaluation, 13 s at 50^3 and 170 s at 80^3 — and supplies every posterior result, Tables 6 and 8, and the 80^3 systems comparison. The *earlier* configuration, used by the 50^3 twin and scaling campaigns (the 50^3 points of Fig. 1, Table 5, Fig. 3 right, Fig. F5) and by the theory diagnostics of Appendix A, inferred the cell division rate and motility on normalized $[0, 1]$ priors mapping to $[6\times 10^{-5}, 0.6]$ and $[0, 10^4]$ (true normalized values 0.0499 and 0.2), scored one simulation per evaluation (5.1 s at 50^3 , in-run coefficient of variation 0.10) by a whitened distance over morphological summary features (radial density and pair-correlation profiles, free-area and structure-factor profiles, a DBSCAN dispersion fraction, and log cell count and radius). The screening above found those two parameters confounded, which is why the production configuration replaced them. The systems measurements depend on the configuration only through its runtime distribution, which the predictor takes from the run itself.

Two properties of the stored records are documented in the repository’s bug log and matter for reproduction. The synchronous arm’s parameter columns in the CSV records of the two-parameter runs were written swapped by a column-ordering defect in the record writer (fixed; the stored runs were repaired from pyABC’s own histories and verified against them), and pyABC’s population weights had been dropped by the record builder (fixed; recovered from the same histories for the Cellular Potts runs, unavailable for the older campaigns).

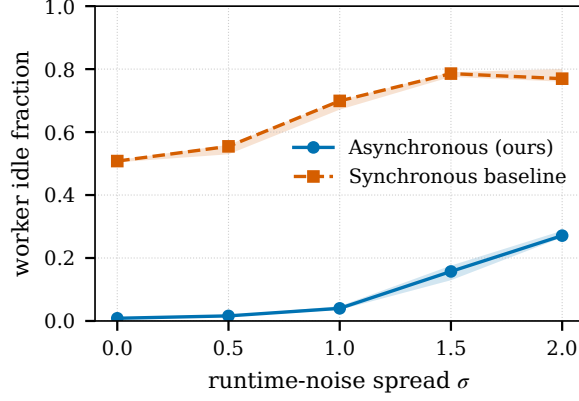


Fig. F1 Worker utilization loss under runtime heterogeneity: the fraction of worker wall-clock spent idle versus the runtime-noise spread σ (five replicates; bands are inter-quartile ranges). The synchronous baseline idles at generation barriers (its idle fraction rises from ≈ 0.51 to ≈ 0.79 as heterogeneity grows) while the asynchronous method stays near-fully utilized (idle fraction ≈ 0.01 rising to ≈ 0.27).

Appendix F Additional results

The studies below support the main text and are reported in full here: the utilisation measurement behind Fig. 5, the parameter-coupled runtime study, strong scaling on the cheap simulator, the calibration sweep over the archive size and snapshot buffer, the ablation and sensitivity grids, and the Lotka–Volterra extinction property.

F.1 Utilisation under runtime heterogeneity

F.2 Parameter-coupled runtime

The heterogeneity study above slows simulations independently of their parameters; a sharper test couples runtime *to the inferred parameter*, directly probing the every-particle bias of the completion-time limitation of §9. On the Gaussian-mean benchmark (48 workers, 60 s budget, five replicates) we set the post-evaluation delay to $d(\mu) = \min(d_0 e^{c(\mu - \mu_c)/\ell}, d_{\max})$ with base $d_0 = 0.02$ s, centre $\mu_c = 0$, length scale $\ell = 0.15$, and cap $d_{\max} = 0.5$ s, and sweep the coupling strength $c \in \{0, 0.5, 1, 2\}$ ($c = 0$ recovers uniform runtime). Larger c makes one region of parameter space evaluate far faster than its mirror image, so the asynchronous “keep every accepted particle” rule over-represents the fast region in the raw archive. The coupling has the intended effect: aggregate throughput falls as it steepens, by $\approx 43\%$ for the asynchronous method from its peak ($\approx 1880/2010 \rightarrow 1150$ sims/s; it is flat between $c = 0$ and $c = 0.5$ and falls monotonically thereafter) and monotonically and more steeply for the barrier-bound baseline ($\approx 840 \rightarrow 250$; Fig. F2a), confirming that the fast region dominates the raw sample counts. Yet the asynchronous posterior-mean error to the analytic Gaussian posterior does not increase as the coupling steepens; it stays ≈ 0.01 and comparable to the discard-latecomers synchronous baseline (Fig. F2b). At the coupling levels tested

the over-representation is measurable in the raw sampling rate but does not measurably distort the reported (unweighted top- k) posterior mean. A targeted ablation that removes the AMIS proposal adaptation ($S=0$) and re-runs the same coupling sweep leaves this robustness essentially unchanged — the asynchronous posterior-mean error stays ≈ 0.005 – 0.013 with or without AMIS and shows no trend in the coupling — which indicates that the raw archive is itself robust at these levels rather than that AMIS actively corrects the over-representation. Two finer tests that the completion-time limitation of §9 motivates sharpen this rather than leave it open. First, we score the AMIS-*reweighted* posterior — the estimator the guarantees of §4 concern, resampling the archive by the retroactive weight (8) — instead of the unweighted archive, and compare it against the no-AMIS variant ($S=0$) across the coupling sweep and a steeper regime (base delay 0.5 s, cap 8 s). The reweighted posterior recovers the analytic mean as accurately as the unweighted archive (absolute error ≤ 0.025 , no trend in the coupling) in both regimes and with or without AMIS; the reweighting’s effect surfaces in the posterior *spread*, not the mean — it lowers the effective sample size (ESS fraction ≈ 0.88 , vs ≈ 0.99 without AMIS), roughly triples the maximum normalised weight (≈ 0.03 vs ≈ 0.01), and widens the central 90% interval by $\approx 15\%$ — so AMIS reshapes the tails but neither creates nor removes a mean bias at these coupling levels. Second, a drain-after-deadline variant lets every in-flight simulation finish past the wall-clock deadline and recomputes the posterior; comparing it against the censored run isolates the completion-time censoring that dividing by the proposal density does not correct. Draining moves the weighted posterior mean by at most 0.013 — comparable to the replicate-to-replicate spread (≈ 0.01) — with no systematic direction, in both the mild and the steep regime. At these coupling levels the deadline censoring is therefore present in principle but negligible in effect, consistent with the sampler concentrating in the fast region near the posterior where few simulations are in flight at the deadline. We accordingly report the raw archive as robust here rather than claiming AMIS corrects the over-representation, and treat a formal censoring correction as future work (the completion-time limitation of §9).

F.3 Strong scaling on a cheap simulator

Lotka–Volterra is deliberately an *adversarial* simulator for a scaling study: each evaluation takes only ≈ 4 ms, so the simulator occupies only a small fraction of each worker’s wall-clock and there is essentially no per-evaluation compute to distribute. A dedicated instrumented run measures this productive fraction directly (Fig. F3): at the 48-worker throughput peak a worker spends only $\approx 25\%$ of its time inside the simulator, leaving $\approx 75\%$ for coordination and idle waiting, and that productive share falls to $\approx 8\%$ and then $\approx 2\%$ as the job spreads from one node to three to six. The mechanism is the method’s single shared population: every completed evaluation is broadcast to all workers so that each can propose from the current archive, an all-to-all exchange whose per-arrival cost grows with the worker count. On a near-instantaneous simulator this exchange saturates and then actively slows the productive ranks. Aggregate throughput therefore rises almost linearly while the job fits on one node ($149 \rightarrow 369 \rightarrow 1983 \rightarrow 6240$ sims/s at $1 \rightarrow 4 \rightarrow 16 \rightarrow 48$ workers) and then *declines* across fully packed multi-node jobs (worker counts that are exact multiples

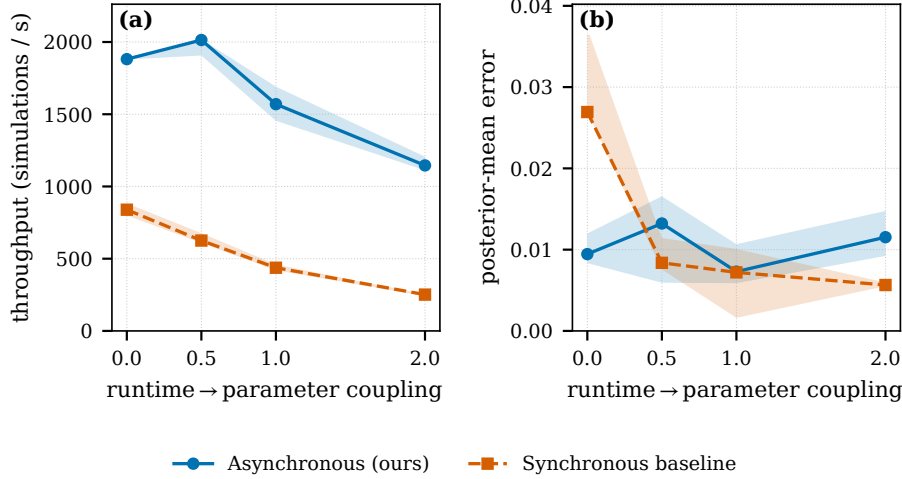


Fig. F2 Parameter-coupled runtime on the Gaussian-mean benchmark (analytic posterior; 48 workers, 60 s budget, five replicates; bands are inter-quartile ranges). The post-evaluation delay grows with the inferred mean, so one region of parameter space is sampled far more often than its mirror image. (a) Aggregate throughput falls as the coupling steepens — monotonically for the barrier-bound baseline, and from a peak at $c = 0.5$ for the asynchronous method — confirming that the fast region dominates the raw sample counts. (b) Posterior-mean error to the analytic posterior nonetheless does not increase with coupling and stays comparable between the asynchronous method and the discard-latecomers synchronous baseline, so the over-representation does not measurably distort the reported posterior at these coupling levels.

of 48): $6240 \rightarrow 4195 \rightarrow 2615 \rightarrow 2088 \rightarrow 1729$ sims/s at $48 \rightarrow 144 \rightarrow 192 \rightarrow 240 \rightarrow 288$ workers, i.e. $1 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$ nodes, because adding ranks beyond one node lengthens the all-to-all exchange faster than it adds useful work. This is a property of coordinating a near-instantaneous simulator across a single shared population, not of asynchrony as such (precisely the regime in which asynchrony has least to offer), and it lifts once the simulator carries real per-evaluation cost that amortizes the coordination (the Cellular Potts result of §7.1; Fig. 3, right). To keep the comparison fair at high core counts we give the synchronous ABC-SMC baseline a *population size equal to the worker count* (a smaller population cannot occupy every core); it then uses all allocated cores and scales steadily, from ≈ 440 sims/s at 16 workers to ≈ 6000 at 288 (Table F3). On this free simulator the asynchronous method leads while the job stays within a few nodes ($\approx 4.5\times$ at 16 workers, $\approx 2\times$ at one full node), but its lead *erodes* with scale and the fair synchronous baseline overtakes it beyond three nodes, where the growing all-to-all coordination cost dominates and the asynchronous method leaves most of the allocation idle. This is the honest boundary of the asynchronous advantage on a cheap, homogeneous simulator; it is largest exactly in the heterogeneous and straggler-prone regimes where barriers otherwise waste compute. The archive size sets where that boundary falls, because the per-arrival proposal reconstruction is $\mathcal{O}(k)$: enlarging the archive tenfold to $k = 1000$ costs throughput at every worker count — 39% at the single-node peak ($6240 \rightarrow 3804$ sims/s) — and brings the crossover with the fair baseline in from beyond three nodes to three (Table F3). Sweeping k across

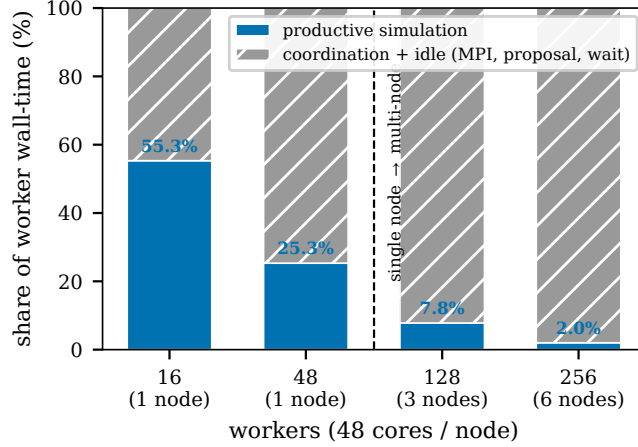


Fig. F3 Measured decomposition of asynchronous worker wall-time on Lotka–Volterra ($k = 100$; 180 s budget) from a dedicated instrumented run. The productive simulation fraction (blue, annotated) — the share of worker wall-clock spent inside the simulator — collapses from 55% at 16 workers to 2% at 256, while coordination and idle (MPI exchange, per-arrival proposal reconstruction, serialization, waiting) dominate exactly as the job spreads beyond one node (16/48 = 1 node; 128 = 3 nodes; 256 = 6 nodes). The strong-scaling decline of Fig. 3 is therefore a measured property of coordinating a near-instantaneous simulator, not an inferred one.

$\{50, 100, 200, 400, 800, 1000\}$ at every worker count resolves that cost into a smooth monotone decline in throughput and a single step in the crossover, which moves in from beyond three nodes to three nodes at $k \geq 400$ (Table F5). The archive size is therefore a systems cost as well as a statistical knob — but, put on the same axis as the calibration it buys, the two turn out *not* to trade off over this range: the best-calibrated archive is also within 7% of the fastest, and every archive larger than $k = 100$ is both slower and worse calibrated (§7.4, Table F5).

F.4 Calibration against the archive size and the snapshot buffer

The calibration tests of §7.4 fix k and S per benchmark, which leaves open whether the two efficiency knobs must be retuned as the dimension grows. To settle this we sweep them directly on a d -dimensional Gaussian-mean target, $y_i \sim \mathcal{N}(\theta, \sigma^2 I_d)$ with $\theta \in \mathbb{R}^d$ under a flat box prior, whose posterior is known exactly at every d (SBC against that exact posterior returns 0.496/0.794/0.905/0.952, confirming the target itself is calibrated, so any departure below is a property of the method). The grid is $d \in \{1, 2, 4, 8, 16\} \times k \in \{50, 100, 200, 400, 800\} \times S \in \{0, 5, 20, 50\}$, 1000 trials per cell, reporting the same full-history estimator (Table F4).

The archive size does not, in this range, trade calibration against throughput.

Because k has both a statistical effect (above) and a systems cost — per-arrival proposal reconstruction is $\mathcal{O}(k)$, so a larger archive is slower at every worker count (§7.3)

— it is natural to expect the two to pull against each other, and to have to buy calibration with throughput. Measuring both on one axis shows that expectation is wrong over the range we can test. Table F5 puts the throughput cost of k next to its worst-case miscalibration over the dimensions swept, using a throughput grid matched on everything but k (Lotka–Volterra, 180s wall cap, five replicates, worker counts 1–288: $k \in \{100, 1000\}$ from the main scaling run and $k \in \{50, 200, 400, 800\}$ from a dedicated k -frontier run sharing that configuration). Of the six archive sizes, only two are Pareto-optimal, and they are the two smallest: $k = 100$ is simultaneously the best-calibrated setting tested (worst deviation 0.064, at $d = 4$) and within 7% of the fastest (6240 vs 6684 simulations/s at the single-node peak). Every larger archive is dominated by it on *both* axes at once — $k = 200$ is 5% slower and nearly three times worse calibrated (0.180), $k = 400$ is 14% slower and 0.195, and $k = 800$ costs 31% of throughput to end up still slightly worse calibrated than $k = 100$ (0.074). The only genuinely faster archive, $k = 50$, is the catastrophic one (0.250 at $d = 8$). So the practical shape of the k decision is not a frontier to negotiate but a floor to clear: go far enough above the small-archive failure region, and then stop, because further enlargement buys no calibration and costs throughput monotonically.

Two qualifications keep this from being read too broadly. The join is on k alone and the two axes come from different benchmarks by design — calibration needs a target whose posterior is known at every d , throughput needs the regime where an $\mathcal{O}(k)$ per-arrival cost is most exposed. That second choice makes the throughput column an *upper* bound on the relative cost of k : Lotka–Volterra evaluations take ≈ 4 ms, so coordination dominates there, and on a cost-bearing simulator the per-evaluation work amortizes the archive operation (§7.3). The conclusion is therefore conservative in the right direction — even where k is most expensive, the best-calibrated archive is nearly free. What k *does* change at scale is where the fair synchronous baseline overtakes the asynchronous method, and there it acts as a step rather than a smooth trade: the crossover sits beyond three fully packed nodes for $k \leq 200$ and moves in to three nodes for $k \geq 400$ (last column).

F.5 The point-mass concentration diagnostic

Earlier versions of this work scored posterior quality by the Wasserstein distance between the unweighted top- k archive and a point mass at the true parameter. That statistic is floored at the posterior’s own spread — on the Gaussian configuration an exactly correct posterior scores $0.1\sqrt{2/\pi} = 0.080$, where every method sat — and it ranks a collapsed archive above a correct one. We retain it only as a concentration diagnostic and report none of its values as posterior quality; a test in the repository pins the property.

F.6 Worker utilisation on Cellular Potts at 50³

F.7 Ablation

This study and the sensitivity grid that follows predate the reported estimator and score the unweighted archive with the point-mass distance of the previous subsection; values below 0.080 do not distinguish a correct posterior from a collapsed one,

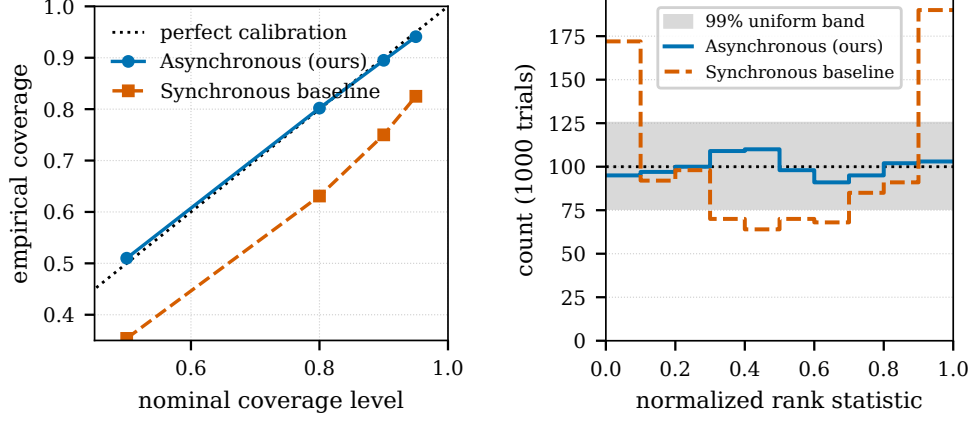


Fig. F4 Simulation-based calibration on Gaussian-mean inference (1000 trials; the reported full-history AMIS posterior): empirical coverage versus nominal level (left) and rank histogram (right). The asynchronous estimator is close to calibrated.

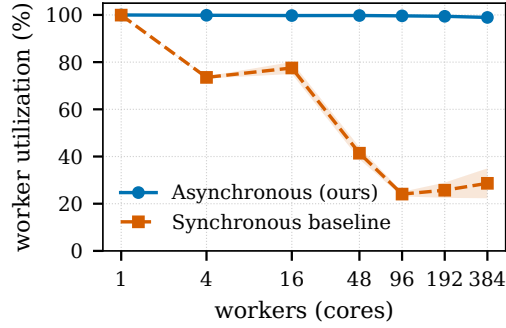


Fig. F5 Cellular Potts 50^3 worker utilization in the benchmark's earlier single-simulation configuration (fraction of worker wall-clock spent inside the simulator) versus worker count (1800 s budget; error bars are ± 1 s.d.); the synchronous baseline uses a population equal to the worker count (Fig. 3, right). The asynchronous method stays ≈ 98.9 – 99.8% utilized at every scale, while the synchronous baseline, even filling every core within a generation, then idles at the barrier waiting for the slowest simulation, so its utilization falls to ≈ 24 – 29% ($41\% \rightarrow 24\% \rightarrow 26\% \rightarrow 29\%$ at 48/96/192/384 workers, i.e. ≈ 70 – 76% idle at every multi-node scale). At this size, however, only $1.2\times$ of the gap is barrier idle (Table 5); the rest is pyABC's per-generation overhead against a 13 s evaluation, which is why the systems claim is made at 80^3 . The figure is the measured utilisation behind the synchronous method's sub-linear scaling in Fig. 3 (right).

so the two studies show that no ingredient removal changes the archive's concentration, and nothing about posterior quality. Reverting the smooth kernel to a hard indicator slightly raises the Wasserstein-to-truth relative to the full method (though within its confidence interval), while removing the AMIS cumulative-mixture reweighting is approximately neutral on this uniform-runtime analytic target (Fig. F6). On the Gaussian-mean benchmark the full method attains a final Wasserstein-to-truth of

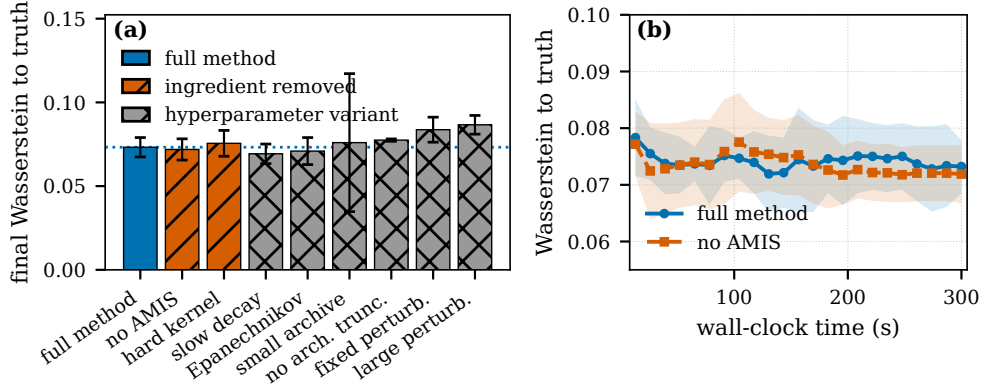


Fig. F6 Ablation on the Gaussian-mean benchmark (analytic posterior; 48 workers, 300s budget, $k = 100$, five replicates, asynchronous method). (a) Final Wasserstein-to-truth for the full method (blue, solid), the two ingredient removals (vermillion, hatched: no AMIS, hard kernel), and hyperparameter variants (grey, cross-hatched); colour and hatch encode the class redundantly, error bars are 95% CIs, and the dotted line marks the full-method mean. (b) AMIS isolation: full method vs. no-AMIS on a shared wall-clock grid (bands are ± 1 s.d. over replicates), zoomed to the converged regime; on this uniform-runtime analytic target the two variants track each other.

0.073 (CI [0.067, 0.079]); the hard-indicator kernel raises it to 0.076 (inside that interval), whereas dropping AMIS leaves it essentially unchanged (0.072; Fig. F6a), with the remaining hyperparameter variants scattered around the full-method reference and degrading only toward the largest perturbation scales (0.084–0.087). That AMIS is neutral here is consistent with the parameter-coupled runtime study above, where removing AMIS likewise left the posterior essentially unchanged: with no runtime–parameter coupling on this uniform-runtime target, there is no over-representation for the reweighting to reshape. The AMIS-isolation trajectory (Fig. F6b) accordingly shows the full method and the no-AMIS variant tracking each other throughout the converged regime (both ≈ 0.072 – 0.073 after 40 s).

F.8 Sensitivity

Across the full hyperparameter grid *on the Gaussian-mean benchmark* (the analytic-posterior sanity model) the asynchronous method’s posterior quality is stable, with no region of catastrophic failure (Fig. F7). The Wasserstein distance to the true parameter stays within ≈ 0.09 – 0.15 across the grid (each cell averaged over five replicates and the two smooth kernels); the archive size k matters only marginally *for point quality at this dimension* (its effect on calibration in higher dimensions is a separate and much larger story, Table F4), and quality degrades only mildly toward the largest perturbation scales and initial tolerances. The tolerance scheduler is inert under a smooth kernel — the asynchronous selector chooses ϵ by ESS-retention bisection regardless of the scheduler type — so we fixed it to the acceptance-rate rule rather than sweeping a variable that cannot change the smooth-kernel result. We therefore did not find delicate tuning necessary to reach the quality reported above *on this benchmark*. We did not

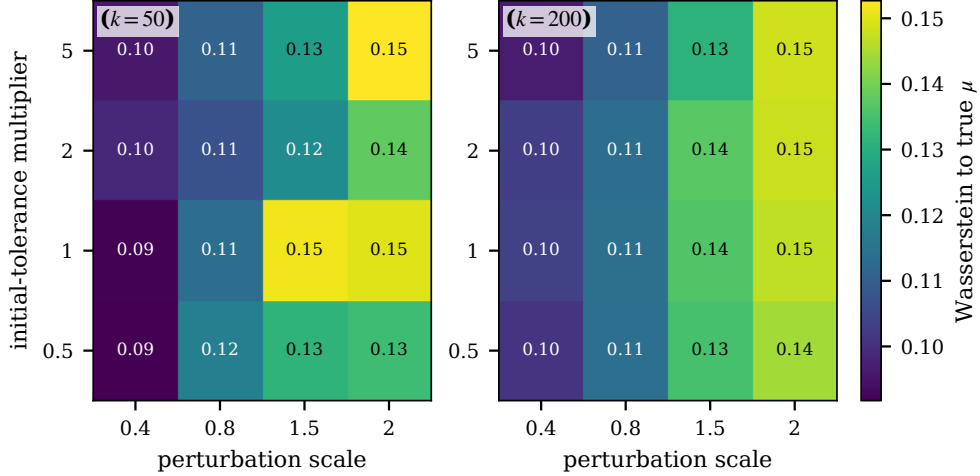


Fig. F7 Hyperparameter sensitivity of the asynchronous method, measured by posterior quality (Wasserstein distance to the true μ , lower is better; averaged over replicates and smoothing kernel). Each panel sweeps perturbation scale (x) against the initial-tolerance multiplier (y); panels are faceted by archive size k . Quality is stable across the grid (Wasserstein ≈ 0.09 – 0.15) with no failure region; the tolerance scheduler is inert under the smooth kernel and is held fixed, while larger perturbation scales degrade quality only mildly.

repeat the full grid on the costlier non-Gaussian benchmarks, so this robustness statement is scoped to the analytic-posterior model and to the qualitative trends observed there (scheduler-insensitive; mild, monotone sensitivity to perturbation scale and initial tolerance) rather than asserted as a general guarantee. Two further scope limits matter. This grid measures *point quality* (Wasserstein to the truth) in *one* dimension; it says nothing about calibration, and the archive size that looks inconsequential here is decisive for coverage in higher dimensions — the dimension-resolved sweep above (Table F4) finds a genuinely catastrophic region ($d = 8$, $k = 50$: coverage 0.29 at the 0.50 level) that a one-dimensional quality grid cannot see. “No failure region” should therefore be read as a statement about one-dimensional point quality only.

F.9 Lotka–Volterra extinction

One property of the Lotka–Volterra benchmark bounds what its statistical numbers can mean, and we state it here rather than leave it to be discovered. The simulator has an absorbing state: if either population reaches zero before $T_{\max}$ the run returns a fallback discrepancy, which a smooth kernel weights to effectively zero. At the configured parameters that is the usual outcome. Survival is 2.6% of 2000 simulations at the true parameter, and 1.7% over 400 uniform prior draws, of which 92.5% never survived one of 25 attempts. The observed dataset is itself survival-conditioned — it is re-drawn until a trajectory survives, which here takes more than eleven attempts — so the target is $p(\theta \mid s_{\text{obs}}, \text{survived})$ against a survival-conditioned s_{obs} , which is internally consistent: the extinct particles’ zero weight *is* the $P(\text{survive} \mid \theta)$ factor of

the likelihood and belongs there. The consequences split. Throughput and scaling on this benchmark are unaffected, because an extinct simulation costs what a surviving one costs and that cost is the systems quantity under test. Posterior recovery is a different matter: the informative sample is $\approx 2\%$ of the nominal budget, so a 1.2×10^6 -evaluation run carries the information of $\approx 25,000$, and any method comparison on it is substantially a comparison of how each copes with a 98%-rejection regime. We therefore report Lotka–Volterra as a systems benchmark and treat its posterior curves as we treat Cellular Potts’s: as diagnostics that bound no method’s attainable accuracy.

References

- [1] Beaumont, M.A.: Approximate bayesian computation. *Annual Review of Statistics and Its Application* **6**, 379–403 (2019) <https://doi.org/10.1146/annurev-statistics-030718-105212>
- [2] Sisson, S.A., Fan, Y., Tanaka, M.M.: Sequential monte carlo without likelihoods. *Proceedings of the National Academy of Sciences* **104**(6), 1760–1765 (2007) <https://doi.org/10.1073/pnas.0607208104>
- [3] Toni, T., Welch, D., Strelkowa, N., Ipsen, A., Stumpf, M.P.H.: Approximate bayesian computation scheme for parameter inference and model selection in dynamical systems. *Journal of the Royal Society Interface* **6**(31), 187–202 (2009) <https://doi.org/10.1098/rsif.2008.0172>
- [4] Beaumont, M.A., Cornuet, J.-M., Marin, J.-M., Robert, C.P.: Adaptive approximate bayesian computation. *Biometrika* **96**(4), 983–990 (2009) <https://doi.org/10.1093/biomet/asp052>
- [5] Del Moral, P., Doucet, A., Jasra, A.: An adaptive sequential monte carlo method for approximate bayesian computation. *Statistics and Computing* **22**(5), 1009–1020 (2012) <https://doi.org/10.1007/s11222-011-9271-y>
- [6] Cornuet, J.-M., Marin, J.-M., Mira, A., Robert, C.P.: Adaptive multiple importance sampling. *Scandinavian Journal of Statistics* **39**(4), 798–812 (2012) <https://doi.org/10.1111/j.1467-9469.2011.00756.x>
- [7] Taubert, O., Weißer, M., Kelle, D., Bayer, M., Knüttel, H., Comito, C., Streit, A., Götz, M.: Massively parallel genetic optimization through asynchronous propagation of populations. In: *International Conference on High Performance Computing (ISC High Performance)*, pp. 106–124 (2023). https://doi.org/10.1007/978-3-031-32041-5_6
- [8] Alamoudi, E., Reck, F., Bundgaard, N., Graw, F., Brusch, L., Hasenauer, J., Schälte, Y.: A wall-time minimizing parallelization strategy for approximate Bayesian computation. *PLOS ONE* **19**(2), 0294015 (2024) <https://doi.org/10.1371/journal.pone.0294015>

- [9] Wilkinson, R.D.: Approximate bayesian computation (abc) gives exact results under the assumption of model error. *Statistical Applications in Genetics and Molecular Biology* **12**(2), 129–141 (2013) <https://doi.org/10.1515/sagmb-2013-0010>
- [10] Fearnhead, P., Prangle, D.: Constructing summary statistics for approximate bayesian computation: semi-automatic approximate bayesian computation. *Journal of the Royal Statistical Society: Series B* **74**(3), 419–474 (2012) <https://doi.org/10.1111/j.1467-9868.2011.01010.x>
- [11] Drovandi, C.C., Pettitt, A.N.: Estimation of parameters for macroparasite population evolution using approximate bayesian computation. *Biometrics* **67**(1), 225–233 (2011) <https://doi.org/10.1111/j.1541-0420.2010.01410.x>
- [12] Lenormand, M., Jabot, F., Deffuant, G.: Adaptive approximate bayesian computation for complex models. *Computational Statistics* **28**(6), 2777–2796 (2013) <https://doi.org/10.1007/s00180-013-0428-3>
- [13] Veach, E., Guibas, L.J.: Optimally combining sampling techniques for monte carlo rendering. In: *Proceedings of SIGGRAPH '95*, pp. 419–428 (1995). <https://doi.org/10.1145/218380.218498>
- [14] Marin, J.-M., Pudlo, P., Sedki, M.: Consistency of adaptive importance sampling and recycling schemes. *Bernoulli* **25**(3), 1977–1998 (2019) <https://doi.org/10.3150/18-BEJ1042>
- [15] Cappé, O., Guillin, A., Marin, J.-M., Robert, C.P.: Population Monte Carlo. *Journal of Computational and Graphical Statistics* **13**(4), 907–929 (2004) <https://doi.org/10.1198/106186004X12803>
- [16] Bugallo, M.F., Elvira, V., Martino, L., Luengo, D., Míguez, J., Djurić, P.M.: Adaptive importance sampling: The past, the present, and the future. *IEEE Signal Processing Magazine* **34**(4), 60–79 (2017) <https://doi.org/10.1109/MSP.2017.2699226>
- [17] Vergé, C., Dubarry, C., Del Moral, P., Moulines, E.: On parallel implementation of sequential Monte Carlo methods: The island particle model. *Statistics and Computing* **25**(2), 243–260 (2015) <https://doi.org/10.1007/s11222-013-9429-x>
- [18] Paige, B., Wood, F., Doucet, A., Teh, Y.W.: Asynchronous anytime sequential monte carlo. In: *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27 (2014)
- [19] Murray, L.M., Lee, A., Jacob, P.E.: Parallel resampling in the particle filter. *Journal of Computational and Graphical Statistics* **25**(3), 789–805 (2016) <https://doi.org/10.1080/10618600.2015.1062015>

- [20] Klinger, E., Rickert, D., Hasenauer, J.: pyabc: distributed, likelihood-free inference. *Bioinformatics* **34**(20), 3591–3593 (2018) <https://doi.org/10.1093/bioinformatics/bty361>
- [21] Schälte, Y., Klinger, E., Stapor, P., Hasenauer, J.: pyabc 0.12.3: high-performance distributed, likelihood-free inference. *Bioinformatics* **38**(13), 3354–3355 (2022) <https://doi.org/10.1093/bioinformatics/btac321>
- [22] Dutta, R., Schoengens, M., Pacchiardi, L., Ummadisingu, A., Widmer, N., Künzli, P., Onnela, J.-P., Mira, A.: Abcpy: A high-performance computing perspective to approximate bayesian computation. *Journal of Statistical Software* **100**(7), 1–38 (2021) <https://doi.org/10.18637/jss.v100.i07>
- [23] Lintusaari, J., Vuollekoski, H., Kangasrääsiö, A., Skytén, K., Järvenpää, M., Marttinen, P., Gutmann, M.U., Vehtari, A., Corander, J., Kaski, S.: ELFI: Engine for likelihood-free inference. *Journal of Machine Learning Research* **19**(16), 1–7 (2018)
- [24] Owen, A., Zhou, Y.: Safe and effective importance sampling. *Journal of the American Statistical Association* **95**(449), 135–143 (2000) <https://doi.org/10.1080/01621459.2000.10473909>
- [25] Berghoff, M., Rosenbauer, J., Hoffmann, F., Schug, A.: Cells in silico – introducing a high-performance framework for large-scale tissue modeling. *BMC Bioinformatics* **21**, 436 (2020) <https://doi.org/10.1186/s12859-020-03728-7>
- [26] Hall, P., Heyde, C.C.: *Martingale Limit Theory and Its Application*. Academic Press, New York (1980)

Table B1 What the implementation guarantees, what it assumes, and what it violates. “By construction” means the code enforces the property for every run, not that it was checked post hoc.

Assumption	Status	Basis
1 kernel, Lipschitz part	by construction	Gaussian and Epanechnikov are peak-normalized and Lipschitz in ϵ on every $[a, \epsilon_0]$, with $L_K(a) = 2/ea$ and $2/a$. The hard indicator is not, and is excluded from the theory.
1 $Z_{\epsilon_\infty} > 0$	holds for Gaussian	The Gaussian kernel is strictly positive, so $L_{\epsilon_\infty} > 0$ and the limiting target is non-degenerate whatever the discrepancy distribution. For a compactly supported kernel (Epanechnikov) it requires the achievable discrepancies to reach inside the support, which we do not verify.
2 denominator class	by construction	The prior support is a box and the archive lies in it, so component means do; the covariance carries an additive jitter $10^{-9}\text{tr}(\widehat{\Sigma})/d + 10^{-12}\bar{b}^2$ ($\bar{b}^2$ the mean squared box width), whose second term is a run-independent floor, giving $\sigma_-^2 > 0$, and σ_+^2 follows from the archive lying in a bounded box. The truncation <i>normalizer</i> is the product of per-axis in-box masses, so with a correlated covariance $\bar{q}_n$ does not integrate to one — which Assumption 2 does not require: it constrains a positive function, and the discrepancy is carried by r instead (§A.1(c)).
4 adapted filtration	assumed	Automatic for a sequential run. Under asynchrony a discrepancy may be in flight when the next candidate is proposed, so the field of realized outcomes cannot serve; the assumption is that a larger filtration exists under which the candidate’s conditional law is still $\bar{q}_i$. Where completion times steer scheduling, $\bar{q}_i$ is that enlarged-field law rather than the emitted proposal density. Same mechanism as the completion-time selection limitation of §9.
3 prior floor	by construction	The retroactive denominator floors the prior component’s mass at $\delta = 0.5/(m+1)$ (Appendix C), so $\bar{q}_n \geq \delta\pi$ and Lemma 6 bounds every reported weight by $1/\delta$ — independently of what the sampler does.
5(i) fidelity (9)	assumed; two of its four factors measured	The one place the sampler enters for Theorem 1. Its four factors are decomposed in §A.1; two of them — the prior floor and the snapshot quadrature — are estimated from stored histories rather than bounded ($\widehat{\zeta} = 0.022$ on the earlier-configuration Cellular Potts run, the reported posterior 1.1% in total variation from a large- m reference), while the truncation constant and the rejection/redraw conditioning are bounded only. Since the factors multiply, the measured part is a partial accounting rather than a bound on the whole.
5(ii) stabilization	violated as configured	Not implied by the top- k rule, and now measured rather than left open: on the earlier-configuration Cellular Potts run the per-step drift decays as $\tau^{-0.92}$ (archive turnover 832 changes in 33,748 draws, the $O(k \log n)$ record rate), while the distance to the limit decays only as τ^{-b} , $b \approx 0.14$ – 0.33 , against the $b > \frac{1}{2}$ needed. A serial benchmark passes at $b \approx 0.6$ – 0.76 when the target is identified and fails at $b \approx 0.0$ – 0.29 when its parameters confound, which is the Cellular Potts case by construction — so the condition is one on the inference problem, not on the sampler. The central limit theorem therefore covers the damped or frozen variants, not this configuration; consistency, needing no rate, is unaffected. The asynchronous replay of the proposal sequence enters here too, since a reporting rank’s arrival-ordered log need not reproduce the proposal each candidate came from.
5 $\epsilon_\infty > 0$ (deterministic, for the CLT only)	not satisfied	A strictly positive <code>min.tol</code> forces $\epsilon_\infty \geq \text{min.tol} > 0$, since $\{\epsilon_n\}$ is non-increasing and clipped from below, which is all Theorem 1 needs. Theorem 3 additionally needs ϵ_∞ deterministic, so that $f - \pi_{\epsilon_\infty}(f)$ is a fixed function; a floor alone does not give that, since a monotone data-driven schedule may settle above it at a run-dependent value. A schedule that reaches its floor in finite time does, and makes the ϵ terms of (10) vanish identically. The runs reported here leave <code>min.tol</code> unset (zero is accepted), so neither part is <i>established</i> — not that either is disproved: a data-driven decreasing schedule may well converge to a positive limit, and may even converge to a deterministic one; we simply do not verify it. Note also that the bandwidth a finite budget reaches is not itself an asymptotic ϵ_∞ .
5 rate (10)	assumed	Needed only in Step 1 and Step 2 of the proof of Theorem 3; consistency does not use it.

Table D2 Resolved experiment configurations. Workers, per-run wall-clock budget, replicates (trials for SBC), archive size k , and methods (A: asynchronous; S: matched-kernel synchronous baseline; R: rejection ABC; T: barrierized twin). The twin row covers the straggler, heterogeneity and Cellular Potts 50³ studies at 16, 48 and 48–384 workers.

[†] Simulation-limited: the budget is per-trial (SBC) or per-run (kill-and-resume) *simulations*, not wall-clock seconds; the twin, and the asynchronous re-run matched to it, stop at a fixed simulation count per level (§6.3, Table 3). [‡] Also run with AMIS on/off, a drain-after-deadline variant, and a steeper coupling regime (base delay 0.5 s, cap 8 s). [§] The Cellular Potts rows of these studies ran the benchmark’s earlier configuration (Appendix E).

Experiment	Workers	Budget (s)	Reps	k	Methods
Gaussian mean	48	300	5	100	A,S,R
g-and-k	48	600	5	100	A,S,R
Lotka–Volterra	48	600	5	100	A,S
Cellular Potts 50 ³	48	3600	5	100	A,S,R
Cellular Potts 80 ³	48	3600	3	100	A,S,R
Straggler	16	300	5	100	A,S
Runtime heterogeneity	48	60	5	100	A,S
Parameter-coupled runtime [‡]	48	60	5	100	A,S
Strong scaling (LV)	1–288	180	5	100/1000	A,S
Strong scaling (CPM) [§]	1–384	1800	3–5	100	A,S
Barrierized twin [§]	16 / 48 / 48–384	sims [†]	5	100	T,A
Sensitivity	48	300	5	50/200	A
Ablation	48	300	5	100	A
SBC (Gaussian, 1-D)	48	20k [†]	1000	100	A,S
SBC (bimodal)	48	20k [†]	1000	100	A,S
SBC (g-and-k, 4-D)	48	40k [†]	1000	200	A,S
SBC (g-and-k, 2×)	48	80k [†]	500	200	A,S
Kill-and-resume	4	120k [†]	1	100	A

Table F3 Strong scaling on Lotka–Volterra: median simulation throughput (sims/s) at a 180 s budget, over five replicates (Fig. 3, left, shows the inter-quartile spread). Asynchronous (ours) at both archive sizes vs. synchronous baseline. Multi-node points use *fully packed* nodes (worker counts that are exact multiples of 48); at counts above 100 the synchronous baseline is given *population size equal to the worker count* so it can occupy every core (a population of 100 leaves the surplus idle), which is why it keeps scaling here. The asynchronous method peaks on one node and then declines as multi-node coordination dominates, so the baseline overtakes it beyond three nodes at $k = 100$. Enlarging the archive tenfold costs throughput at every scale — the per-arrival proposal reconstruction is $\mathcal{O}(k)$ — and moves that crossover in from beyond three nodes to three.

Workers	1	4	16	48	144	192	240	288
Nodes	<1	<1	<1	1	3	4	5	6
Synchronous	105	96	437	2952	3757	4322	4539	6034
Asynchronous ($k = 100$)	149	369	1983	6240	4195	2615	2088	1729
Asynchronous ($k = 1000$)	106	133	620	3804	2804	1957	1587	1336

Table F4 Calibration sensitivity to the archive size k and the AMIS snapshot buffer S on the d -dimensional Gaussian-mean target (1000 trials per cell; entries are the mean signed deviation of empirical from nominal coverage over the 0.50/0.80/0.90/0.95 levels and the d parameters). Negative is over-confident, positive is conservative; 0.000 is exactly calibrated. **Left:** varying k at $S = 20$. **Right:** varying S at $k = 100$.

	archive size k (at $S = 20$)				
	50	100	200	400	800
$d = 1$	-0.014	-0.003	+0.006	+0.006	+0.074
$d = 2$	-0.020	-0.006	+0.009	+0.030	+0.072
$d = 4$	+0.025	+0.064	+0.094	+0.133	+0.063
$d = 8$	-0.250	+0.055	+0.180	+0.195	+0.061
$d = 16$	-0.047	+0.028	+0.049	+0.053	+0.055

	snapshot buffer S (at $k = 100$)			
	0	5	20	50
$d = 1$	-0.088	-0.001	-0.003	-0.005
$d = 2$	-0.088	-0.001	-0.006	-0.003
$d = 4$	+0.039	+0.068	+0.064	+0.064
$d = 8$	+0.065	+0.006	+0.055	+0.074
$d = 16$	-0.010	+0.010	+0.028	+0.034

Table F5 The archive size on both of its axes at once. **Throughput:** median asynchronous throughput at the single-node peak (48 workers) on Lotka–Volterra, 180 s wall cap, five replicates ($k \in \{100, 1000\}$ from the main scaling run, $k \in \{50, 200, 400, 800\}$ from the matched k -frontier run), and its change relative to the $k=100$ we use throughout. **Calibration:** worst absolute deviation of empirical from nominal coverage over the dimensions of the d -dimensional Gaussian-mean sweep at $S=20$ (Table F4), with the dimension attaining it. **Crossover:** fully packed nodes at which the fair synchronous baseline (population = worker count) overtakes the asynchronous arm. Only $k=50$ and $k=100$ are Pareto-optimal; every larger archive is both slower *and* worse calibrated than $k=100$. The sweep did not include $k=1000$.

k	sims/s at $W=48$	vs. $k=100$	worst $ \Delta\text{cov} $	at	baseline overtakes
50	6684	+7%	0.250	$d=8$	beyond 3 nodes
100	6240	—	0.064	$d=4$	beyond 3 nodes
200	5904	−5%	0.180	$d=8$	beyond 3 nodes
400	5393	−14%	0.195	$d=8$	3 nodes
800	4331	−31%	0.074	$d=1$	3 nodes
1000	3804	−39%	—	—	3 nodes